

THF: Designing Low-Latency Tweakable Block Ciphers

Jianhua Wang¹, Tao Huang², Guang Zeng³, Tianyou Ding, Shuang Wu², and Siwei Sun¹

¹ School of Cryptology, University of Chinese Academy of Sciences, Beijing, China

² Shield Lab, Huawei International Pte. Ltd., Singapore

³ Shield Lab, Huawei Technologies Co., Ltd., China

Abstract. We introduce a concrete instance of the LRW+ paradigm: the **Three-Hash Framework (THF)**, a mode for constructing tweakable block ciphers that employs three hash functions to process tweak inputs. Through a general practical cryptanalysis of **THF** in both single-tweak and multiple-tweak settings, and by comparing it with two representative modes, **4-LRW1** and **2-LRW2**, we demonstrate that **THF** has the potential to achieve lower latency due to its more balanced resistance to both single- and multiple-tweak attacks. Based on this framework, we design **Blink**, a family of tweakable block ciphers specifically optimized for minimal latency. The hash functions in **Blink** were carefully chosen to align with the low-latency requirements, ensuring both efficiency and strong security. A thorough cryptanalysis of **Blink** is provided, with an emphasis on its resistance to multiple-tweak attacks. Finally, hardware evaluations highlight the exceptional low-latency performance of **Blink**, distinguishing it as one of the most efficient tweakable block ciphers.

Keywords: Low latency · THF · Hash function · Blink.

1 Introduction

Symmetric-key cryptography plays a vital role in securing information across a wide array of applications, including communication protocols, financial systems, embedded devices, and cloud services. Over the years, the field has seen significant advancements, transitioning from general-purpose cryptographic designs to solutions tailored for specific domains. This evolution reflects the increasing demand for cryptographic primitives optimized for the unique requirements of various application scenarios.

Various memory protection solutions have been proposed to enhance the security of data stored in memory, such as Intel Software Guard Extensions (SGX) [29], Intel Trust Domain Extensions (TDX) [33], AMD Secure Encrypted Virtualization (SEV) [3], and ARM Confidential Compute Architecture (CCA) [5]. In these schemes, a symmetric-key primitive is employed as the core cryptographic mechanism within the Memory Encryption Engine (MEE), which is implemented on the System on Chip (SoC) between the DRAM Controller and the system

caches. The MEE plays a vital role in ensuring memory security by decrypting data when it is loaded from DRAM and verifying its integrity to detect potential tampering or corruption. Similarly, when data is written to the DRAM, the MEE encrypts the data and generates an integrity tag in certain solutions such as TDX, which may be stored in the DRAM (either in the memory itself or by repurposing the ECC bits in DRAM).

However, the integration of the MEE in memory-intensive applications introduces additional overhead due to the encryption and decryption operations it performs. To maintain system performance and meet the real-time requirements of such applications, it is essential to minimize this latency. Therefore, the design of low-latency block ciphers becomes a key consideration in the development of memory protection solutions, as these ciphers must minimize the performance impact of the MEE while ensuring robust security.

Tweakable block ciphers [46] enhance the flexibility of cryptographic algorithms by introducing a public tweak as an additional input, which allows each tweak to correspond to a different instantiation of the block cipher. This added flexibility significantly strengthens the adaptability of the algorithm for various applications. In the context of memory encryption, tweakable block ciphers can address specific challenges, such as the ciphertext repetition problem inherent in the ECB mode, by incorporating physical addresses or random nonces as tweaks.

There are two main approaches for designing tweakable block ciphers. One approach is based on modular methods, where tweakable block ciphers are constructed by extending traditional block cipher modes. Examples include LRW1 [46], LRW2 [46], and TNT [10] and so on [40,37,38,35]. The other approach is the TWEAKEY method [34], which treats the tweak and key as a unified role. This method allows for the use of a single key schedule for both the key and tweak, streamlining the cipher’s design. Many cryptographic instances have adopted the TWEAKEY method, including SKINNY [14], MANTIS [14], and QARMA [6] and so on.

The design of low-latency cryptographic primitives is a relatively new and rapidly evolving research field, driven by the demand for efficient encryption in performance-critical applications. Among the earliest low-latency block ciphers is PRINCE [20], introduced in 2012 and specifically optimized for hardware latency. Building on its foundation, an updated version, PRINCEv2 [21], was proposed in 2020, offering enhanced security properties while maintaining its latency-optimized design.

Several other low-latency ciphers have followed. For example, MANTIS [14] provides a 64-bit block size, combining TWEAKEY principles with low-latency properties. Similarly, the QARMA family [6] draws inspiration from both PRINCE and MANTIS, offering low-latency performance with support for 64-bit and 128-bit block sizes. Its updated variant, QARMAv2 [7], introduces enhancements such as support for larger tweaks and adaptations for applications like pointer authentication and memory encryption.

Other innovations in low-latency cipher design include SPEEDY [44], a family of block ciphers optimized specifically for latency. SPEEDY leverages a low-

latency 6-bit S-box and employs a 192-bit block size. For cache randomization, SCARF [22] adopts an extremely compact 10-bit block size combined with a 240-bit key, achieving ultra-low latency. In the context of pointer authentication, BipBip [15] introduces a low-latency tweakable block cipher tailored to Intel’s C^3 architecture. Notably, BipBip incorporates several innovative features such as a nonlinear tweak and key schedule, heterogeneous rounds, and a large master key, showcasing its adaptability to modern cryptographic requirements. Hu *et al.* proposed the uKNIT [32] design framework, which enables fine-grained, round-wise optimization of cryptographic primitives by allowing each round to employ distinct component structures. As a concrete instantiation of this framework, they designed the block cipher uKNIT-BC. Belkheyar *et al.* also introduced CHILOW [16], a family of tweakable block ciphers and a related pseudorandom function (PRF) specifically designed for embedded code encryption. At the core of CHILOW lies CHICHI, a novel family of nonlinear layers defined over even dimensions and constructed based on the well-known χ function.

In the realm of pseudo-random functions (PRFs), the low-latency design strategy of Orthros [9], which uses branches of permutations to form a PRF, inspired the development of Gleeok [4]. This recent PRF supports a 256-bit key size, enhancing its applicability to cryptographic schemes requiring high security and efficiency. Furthermore, Twinkle [58] is a PRF designed with a large, tailored internal state, optimized to achieve low latency and minimal area overhead, making it highly suitable for authenticated encryption schemes in memory protection scenarios.

1.1 Motivation

Existing designs of low-latency tweakable block ciphers, such as MANTIS, QARMA, and QARMAv2, have made strides in optimizing round functions to minimize latency. However, through the security analyses of these designs, we have identified that their resistance to multiple-tweak differential attacks has become a major bottleneck for further latency reduction. This limitation primarily stems from the use of linear TWEAKKEY schedules, which offer limited diffusion of tweak differences. Moreover, the injection of tweak-related round keys in every round increases the likelihood of differential cancellation, further weakening the cipher’s resilience against multiple-tweak differential attacks.

This observation naturally leads us to ask: Can we improve the resistance to multiple-tweak differential attacks by redesigning or optimizing the tweak schedule? Moreover, we investigate whether an improved tweak schedule can narrow the gap between the number of rounds required to resist multiple-tweak attacks and those needed for single-tweak security. If such a design is possible, it would imply that the number of rounds required to meet security goals in the multiple-tweak setting could approach that in the single-tweak setting. This, in turn, opens up a promising opportunity: to construct a tweakable block cipher with similar latency to an existing low-latency block cipher.

1.2 Our Contributions

Based on the above motivation, we develop a new mode for constructing tweakable block ciphers that significantly enhances resistance to multiple-tweak attacks. Inspired by the "prove-then-prune" methodology commonly adopted in modern symmetric cryptographic design (e.g., [49,31,10]), we first establish the theoretical security of the mode and then instantiate a practical, low-latency cipher within this framework.

Our main contributions are summarized as follows:

Three-Hash Framework. We present a new mode for constructing tweakable block ciphers based on the LRW+ paradigm [35], called the **Three-Hash Framework (THF)**. This mode integrates three pairwise independent hash functions to process the tweak. Given a plaintext $\mathbf{m}$ and a tweak $\mathbf{t}$, the encryption is defined as:

$$\mathbf{c} = \pi_4 \left(\pi_3 \left(\pi_2 \left(\pi_1(\mathbf{m}) \oplus h_1(\mathbf{t}) \right) \oplus h(\mathbf{t}) \right) \oplus h_2(\mathbf{t}) \right),$$

where $\pi_1, \pi_2, \pi_3, \pi_4$ are permutations and h_1, h, h_2 are hash functions.

We formally prove that THF achieves beyond-birthday-bound (BBB) security under standard cryptographic assumptions. To further evaluate its robustness in practical scenarios, we extend the *general practical cryptanalysis* framework [27] to multiple-tweak attack settings. Through this extended analysis, we demonstrate that, compared to two representative BBB-secure LRW+ modes—4-LRW1 and 2-LRW2—THF offers a more balanced resistance to both single-tweak and multiple-tweak attacks. This advantage primarily stems from the layered construction of THF, which enables partial parallelism: h_1 and π_1 can be computed concurrently during encryption, while h_2 and π_4^{-1} can be processed in parallel during decryption.

Design of a Low-Latency Tweakable Block Cipher. We instantiate THF with **Blink**, a tweakable block cipher family specifically designed to minimize latency. The main design features include:

- *Logarithmic-depth hashing:* **Blink** adopts a hash function family based on Toeplitz matrices, enabling efficient tweak diffusion with circuit depth that grows only logarithmically with the tweak size. Compared to conventional linear tweak-processing methods, this hash construction significantly enhances resistance against multiple-tweak differential and linear attacks, offering exponential security improvements.
- *Latency-optimized round function:* The round function is inspired by MIDORI, using the same diffusion matrix but replacing the S-boxes with lower-latency alternatives and applying optimized cell permutations. This ensures high security while keeping the round function efficient.
- *Long master key:* Instead of implementing a conventional key schedule, **Blink** directly derives round keys by slicing a long master key. This approach simplifies encryption and decryption routines, reduces the need to repeatedly

invoke a key schedule function, and enhances performance. Additionally, the longer key length improves the cipher’s resistance against brute-force attacks and contributes to reduced latency.

We present a thorough cryptanalysis of **Blink**, particularly in the context of multiple-tweak attacks. Our results show that **Blink**, built atop **THF**, achieves robust security with fewer rounds than comparable designs. We further perform hardware evaluations that confirm **Blink**’s superior latency performance. These results validate the design principles behind **THF**, confirming its potential as a foundation for constructing ultra-low-latency tweakable block ciphers.

Organization. The remainder of this paper is organized as follows. Section 2 introduces the necessary preliminaries. Section 3 describes the **THF** framework and formally proves its security bounds. Section 4 explains the rationale for adopting the **THF** framework in designing low-latency tweakable block ciphers, supported by general practical cryptanalysis. In Section 5, we present the instantiation of **THF** through the cipher **Blink**. Section 6 elaborates on the design rationale and core principles behind **Blink**. A comprehensive security analysis is provided in Section 7. Section 8 evaluates the hardware performance of **Blink**. Finally, Section 9 concludes the paper.

2 Preliminaries

Let $\mathbf{x}$ represent a string and x_i the i -th bit of x . The set $\{0, 1\}^n$ consists of all binary strings of length n , and $\text{Perm}(n)$ denotes the set of all permutations over $\{0, 1\}^n$. For any finite set $\mathcal{X}$, $|\mathcal{X}|$ represents its cardinality, and $x \leftarrow_{\$} \mathcal{X}$ indicates that x is sampled uniformly at random from $\mathcal{X}$.

2.1 Tweakable Block Cipher

A tweakable block cipher [46] extends the functionality of a standard block cipher by introducing an additional input called the *tweak*. Unlike a traditional block cipher, which only takes a key and a plaintext to produce a ciphertext, a tweakable block cipher incorporates the tweak as a third input. Formally, it takes the key, tweak, and plaintext as inputs and outputs the ciphertext.

For a fixed value of the tweak, a tweakable block cipher can be viewed as a standard block cipher, where the tweak effectively modifies the encryption function without changing the key. This added flexibility allows tweakable block ciphers to support diverse cryptographic applications, such as disk encryption, where the tweak can represent sector or block indices to ensure ciphertext uniqueness even for an identical plaintext encrypted under the same key.

2.2 Some Special Function Families

A (τ, n) -hash function family $\mathcal{H}$ is defined as a set of functions $\{h : \{0, 1\}^\tau \rightarrow \{0, 1\}^n\}$. Moreover, a (τ, n) -hash function family $\mathcal{H}$ is called an ϵ -almost XOR universal hash family (AXUHF) if for all distinct $\mathbf{t}, \mathbf{t}' \in \{0, 1\}^\tau$ and $\boldsymbol{\theta} \in \{0, 1\}^n$,

$$\Pr(h \leftarrow_{\$} \mathcal{H} : h(\mathbf{t}) \oplus h(\mathbf{t}') = \boldsymbol{\theta}) \leq \epsilon.$$

For the special case of $\boldsymbol{\theta} = 0^n$, $\mathcal{H}$ is referred to as an ϵ -AUHF.

A (τ, n) -tweakable permutation family $\tilde{\mathcal{H}}$ is a family of functions $\{\tilde{h} : \{0, 1\}^\tau \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ and $\forall \mathbf{t} \in \{0, 1\}^\tau, \tilde{h}(\mathbf{t}, \cdot) \in \text{Perm}(n)\}$. $\tilde{\mathcal{H}}$ is an ϵ -almost universal tweakable permutation family (AUTPF) if and only if for all distinct $(\mathbf{t}, \mathbf{m}), (\mathbf{t}', \mathbf{m}') \in \{0, 1\}^\tau \times \{0, 1\}^n$,

$$\Pr(\tilde{h} \leftarrow_{\$} \tilde{\mathcal{H}} : \tilde{h}(\mathbf{t}, \mathbf{m}) = \tilde{h}(\mathbf{t}', \mathbf{m}')) \leq \epsilon.$$

2.3 Implementation of Boolean Functions

To mathematically model the latency of Boolean circuits for specific applications, we adopt the notion of *depth* as defined in [8].

Definition 1 (Depth). *The depth is defined as the sum of the sequential path delays of basic operations, namely AND, OR, NAND, NOR, and NOT.*

Following the assumption in [8], we assign depth weights to basic logic operations as follows: XOR/XNOR are assigned a weight of 2, AND/OR a weight of 1.5, NAND/NOR a weight of 1, and NOT a weight of 0.5. This model serves as a practical initial guideline for estimating circuits' latency before conducting detailed hardware evaluations. The final latency figures will be obtained through evaluation using a standard cell library.⁴

2.4 Known Attacks on Symmetric Primitives

Differential Attacks. The differential probability between an input difference $\boldsymbol{\delta}_{\text{in}}$ and an output difference $\boldsymbol{\delta}_{\text{out}}$ for a function F is defined as

$$\Pr(\boldsymbol{\delta}_{\text{in}} \xrightarrow{F} \boldsymbol{\delta}_{\text{out}}) = \frac{|\{x \in \{0, 1\}^n \mid F(x) \oplus F(x \oplus \boldsymbol{\delta}_{\text{in}}) = \boldsymbol{\delta}_{\text{out}}\}|}{2^n}.$$

The differential trail of a cipher $E_k = F_r \circ \dots \circ F_1$ is represented by the sequence $(\boldsymbol{\delta}_0, \boldsymbol{\delta}_1, \dots, \boldsymbol{\delta}_r)$, where each $\boldsymbol{\delta}_i$ denotes the intermediate difference after applying

⁴ Although, as shown in [44], the actual latency of basic operations may vary depending on multiple factors, which can lead to deviations from the assigned depth weights, this measure remains highly valuable in cryptographic design. Many designs with small S-boxes based on this metric—such as those in Midori [8] and Orthros [9]—have demonstrated excellent hardware performance. Thus, we use these weights as initial design references, with final delay evaluations performed using a standard cell library.

round function F_i . Assuming the independence of round keys, the probability of this trail is given by

$$\prod_{i=1}^r \Pr(\delta_{i-1} \xrightarrow{F_i} \delta_i).$$

The overall differential probability from input difference δ_0 to output difference δ_r under the cipher E_k is then computed by summing over all possible intermediate trails:

$$\Pr(\delta_0 \xrightarrow{E_k} \delta_r) = \sum_{(\delta_1, \dots, \delta_{r-1})} \prod_{i=1}^r \Pr(\delta_{i-1} \xrightarrow{F_i} \delta_i).$$

Linear Attacks. The linear correlation between an input mask λ_{in} and an output mask λ_{out} of a function F is defined as

$$\text{Cor}(\lambda_{in} \xrightarrow{F} \lambda_{out}) = 2 \Pr(\lambda_{in}^T \cdot x \oplus \lambda_{out}^T \cdot F(x) = 0) - 1,$$

where $\cdot$ denotes the matrix multiplication over $\mathbb{F}_2$.

The linear trail of a cipher $E_k = F_r \circ \dots \circ F_1$ is described by the sequence $(\lambda_0, \lambda_1, \dots, \lambda_r)$, where each λ_i denotes the linear mask after round function F_i . Assuming the independence of round keys, the correlation of the trail is given by

$$\prod_{i=1}^r \text{Cor}(\lambda_{i-1} \xrightarrow{F_i} \lambda_i).$$

The overall linear correlation from input mask λ_0 to output mask λ_r under the cipher E_k is computed by summing over all possible intermediate trails:

$$\text{Cor}(\lambda_0 \xrightarrow{E_k} \lambda_r) = \sum_{\lambda_1, \dots, \lambda_{r-1}} \prod_{i=1}^r \text{Cor}(\lambda_{i-1} \xrightarrow{F_i} \lambda_i).$$

Algebraic Attacks. An n -variable Boolean function f can be uniquely expressed in its algebraic normal form (ANF) as

$$f(x) = \bigoplus_{v \in \mathbb{F}_2^n} a_v x^v, \quad a_v \in \mathbb{F}_2,$$

where $x^v = x_1^{v_1} x_2^{v_2} \dots x_n^{v_n}$.

Given a fixed $u \in \mathbb{F}_2^n$, the function $f(x)$ can be rewritten as

$$f(x) = \sum_{v \preceq u} g_{u,v}(x) \cdot x^v,$$

where

$$g_{u,v}(x) = \bigoplus_{w \preceq u^c} a_{w \oplus v} x^w.$$

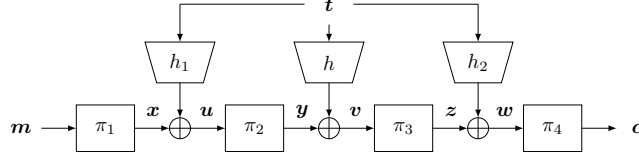


Fig. 1. The diagram of THF mode

Here, $\mathbf{u}^c$ denotes the bitwise complement of $\mathbf{u}$, i.e., the i -th bit of $\mathbf{u}^c$ is $u_i \oplus 1$. The partial order $\mathbf{v} \preceq \mathbf{u}$ indicates that $v_i \leq u_i$ for all i .

The *algebraic degree* of f with respect to the variable subset $\mathbf{x}_\mathbf{u}$ (i.e., those x_i such that $u_i = 1$) is defined as

$$\deg(f, \mathbf{u}) = \max_{\mathbf{v} \preceq \mathbf{u}} \{ \text{wt}(\mathbf{v}) \mid g_{\mathbf{u}, \mathbf{v}}(\mathbf{x}) \neq 0 \},$$

where $\text{wt}(\mathbf{v})$ denotes the Hamming weight of $\mathbf{v}$.

The algebraic degree serves as a key indicator of a Boolean function's resistance to algebraic attacks: higher degree typically implies stronger security.

3 New Scheduling Scheme of Tweak

We propose a new instantiation of the LRW+ mode [35], called the **Three-Hash Framework (THF)**, which employs three hash functions to process the tweak, thereby supporting tweaks of arbitrary length. Unlike **TWEAKEY** framework that treats the key and tweak uniformly, THF deliberately separates their roles, as related-key attacks are outside our threat model. Instead, the design focuses on strengthening resistance to multiple-tweak attacks. In the following, we detail the construction of THF and provide a formal theoretical security analysis.

3.1 The THF Mode

Let $\mathcal{H}$ and $\mathcal{H}'$ be two (τ, n) -hash function families. For $h \in \mathcal{H}$, $h_1, h_2 \in \mathcal{H}'$, and permutations $\pi_i \in \text{Perm}(n)$ for $i \in \{1, \dots, 4\}$, the encryption of a plaintext $\mathbf{m} \in \{0, 1\}^n$ under a tweak $\mathbf{t} \in \{0, 1\}^\tau$ proceeds as follows (illustrated in Figure 1):

$$\mathbf{c} = \pi_4 \left(\pi_3 \left(\pi_2 \left(\pi_1(\mathbf{m}) \oplus h_1(\mathbf{t}) \right) \oplus h(\mathbf{t}) \right) \oplus h_2(\mathbf{t}) \right),$$

where $\mathbf{c} \in \{0, 1\}^n$ is the resulting ciphertext.

3.2 Security of the THF

THF is an instance of the LRW+ construction [35], which is a (τ, n) -tweakable permutation family introduced by Jha *et al.* at EUROCRYPT 2024. Let $\tilde{\mathcal{H}}$ represent a family of (τ, n) -tweakable permutations. The LRW+ construction is defined by the following mapping:

$$\mathbf{c} = \tilde{h}_2^{-1} \left(\mathbf{t}, \pi_2 \left(h(\mathbf{t}) \oplus \pi_1 \left(\tilde{h}_1(\mathbf{t}, \mathbf{m}) \right) \right) \right)$$

where $\widetilde{h_1}, \widetilde{h_2} \leftarrow_{\$} \widetilde{\mathcal{H}}$, $\pi_1, \pi_2 \rightarrow_{\$} \text{Perm}(n)$, $h \leftarrow_{\$} \mathcal{H}$ and for all $\mathbf{t}$, $\widetilde{h_2}^{-1}(\mathbf{t}, \cdot)$ represents the inverse of $\widetilde{h_2}(\mathbf{t}, \cdot)$. The security of LRW+ relies on the property of $\widetilde{\mathcal{H}}$ and $\mathcal{H}$, as outlined in Theorem 1. For more details, please refer to [35].

Theorem 1 ([35]). *Let $0 \leq \epsilon_1, \epsilon_2 \leq 1$. If $\widetilde{\mathcal{H}}$ is an ϵ_1 -AUTPF, and $\mathcal{H}$ is an ϵ_2 -AUHF. Besides, $\widetilde{h_1}$, $\widetilde{h_2}$ and h are pairwise independent. Then, for $q \leq 2^{n-2}$, we have*

$$\text{Adv}_{\text{LRW+}}^{\text{ind-cca}}(q) \leq \epsilon(q, n),$$

where $\epsilon(q, n) = 2q^2\epsilon_1^{1.5} + 2^{2-n}q^4\epsilon_1^2 + 2^{5-2n}q^4\epsilon_1 + 13 \cdot 2^{-3n}q^4 + q^2\epsilon_1^2 + q^2\epsilon_1\epsilon_2 + 2^{1-2n}q^2$.

In the THF construction (see Figure 1), the mappings $\pi_1(\mathbf{m}) \oplus h_1(\mathbf{t})$ and $\pi_4(h_2(\mathbf{t}) \oplus \mathbf{z})$ correspond to the (τ, n) -tweakable permutations $\widetilde{h_1}$ and $\widetilde{h_2}^{-1}$, respectively, in the LRW+ paradigm. Similarly, π_2 , π_3 , and h play the roles of the intermediate permutations and the central hash function, respectively.

Notably, we have $(\pi_4(h_2(\mathbf{t}) \oplus \mathbf{z}))^{-1} = \pi_4^{-1}(\mathbf{c}) \oplus h_2(\mathbf{t})$. Therefore, to analyze the security bounds of THF via Theorem 1, it is crucial to study the properties of the tweakable permutation family

$$\widetilde{\mathcal{H}'} := \{\pi(\mathbf{m}) \oplus h(\mathbf{t}) \mid \pi \in \mathcal{P}, h \in \mathcal{H}'\},$$

where $\mathcal{P} \subseteq \text{Perm}(n)$.

Proposition 1. *If $\widetilde{\mathcal{H}'}$ is an ϵ -AUTPF, then $\mathcal{H}'$ must be an ϵ -AUHF.*

Proof. By the definition of ϵ -AUTPF, we know that for any distinct pairs $(\mathbf{m}, \mathbf{t})$ and $(\mathbf{m}', \mathbf{t}')$, it holds that

$$\begin{aligned} & \Pr(\widetilde{h} \leftarrow_{\$} \widetilde{\mathcal{H}'} : \widetilde{h}(\mathbf{t}, \mathbf{m}) = \widetilde{h}(\mathbf{t}', \mathbf{m}')) \leq \epsilon \\ \Leftrightarrow & \Pr(\pi \leftarrow_{\$} \mathcal{P}, h \leftarrow_{\$} \mathcal{H}' : \pi(\mathbf{m}) \oplus h(\mathbf{t}) = \pi(\mathbf{m}') \oplus h(\mathbf{t}')) \leq \epsilon. \end{aligned}$$

If we fix $\mathbf{m} = \mathbf{m}'$, this simplifies to:

$$\Pr(h \leftarrow_{\$} \mathcal{H}' : h(\mathbf{t}) = h(\mathbf{t}')) \leq \epsilon,$$

which implies that $\mathcal{H}'$ is an ϵ -AUHF.

Proposition 1 indicates that the condition of $\mathcal{H}'$ being an ϵ -AUHF is necessary for $\widetilde{\mathcal{H}'}$ to be an ϵ -AUTPF. Furthermore, we observed that a stronger property of $\mathcal{H}'$ suffices to ensure that $\widetilde{\mathcal{H}'}$ is indeed an ϵ -AUTPF, as stated in Lemma 1.

Lemma 1. *If $\mathcal{H}'$ is an ϵ -AXUHF, then for any non-empty $\mathcal{P} \subseteq \text{Perm}(n)$, $\widetilde{\mathcal{H}'}$ must be an ϵ -AUTPF.*

Proof. For any distinct pairs $(\mathbf{m}, \mathbf{t})$ and $(\mathbf{m}', \mathbf{t}')$, we consider two cases: (1) $\mathbf{t} = \mathbf{t}'$ and (2) $\mathbf{t} \neq \mathbf{t}'$.

- **Case 1:** If $\mathbf{t} = \mathbf{t}'$, then $\mathbf{m} \neq \mathbf{m}'$. Since π is a permutation, it follows that $\pi(\mathbf{m}) \neq \pi(\mathbf{m}')$. Then,

$$\begin{aligned}
& \Pr(\tilde{h} \leftarrow_{\$} \tilde{\mathcal{H}}' : \tilde{h}(\mathbf{t}, \mathbf{m}) = \tilde{h}(\mathbf{t}', \mathbf{m}')) \\
&= \Pr(\pi \leftarrow_{\$} \mathcal{P}, h \leftarrow_{\$} \mathcal{H}' : \pi(\mathbf{m}) \oplus h(\mathbf{t}) = \pi(\mathbf{m}') \oplus h(\mathbf{t}')) \\
&= \Pr(\pi \leftarrow_{\$} \mathcal{P} : \pi(\mathbf{m}) = \pi(\mathbf{m}')) \\
&= 0.
\end{aligned}$$

- **Case 2:** If $\mathbf{t} \neq \mathbf{t}'$, we proceed as follows:

$$\Pr(\tilde{h} \leftarrow_{\$} \tilde{\mathcal{H}}' : \tilde{h}(\mathbf{t}, \mathbf{m}) = \tilde{h}(\mathbf{t}', \mathbf{m}')) \quad (1)$$

$$= \Pr(\pi \leftarrow_{\$} \mathcal{P}, h \leftarrow_{\$} \mathcal{H}' : \pi(\mathbf{m}) \oplus h(\mathbf{t}) = \pi(\mathbf{m}') \oplus h(\mathbf{t}')) \quad (2)$$

$$= \sum_{\boldsymbol{\theta} \in \{0,1\}^n} \Pr(\pi \leftarrow_{\$} \mathcal{P} : \pi(\mathbf{m}) \oplus \pi(\mathbf{m}') = \boldsymbol{\theta}) \cdot \Pr(h \leftarrow_{\$} \mathcal{H}' : h(\mathbf{t}) \oplus h(\mathbf{t}') = \boldsymbol{\theta}) \quad (3)$$

$$\leq \epsilon \cdot \sum_{\boldsymbol{\theta} \in \{0,1\}^n} \Pr(\pi \leftarrow_{\$} \mathcal{P} : \pi(\mathbf{m}) \oplus \pi(\mathbf{m}') = \boldsymbol{\theta}) \quad (4)$$

$$= \epsilon. \quad (5)$$

The inequation (4) follows from the fact that $\mathcal{H}'$ is an ϵ -AXUHF.

Therefore, $\tilde{\mathcal{H}}'$ meets the requirements of an ϵ -AUTPF.

Therefore, the security bound of the THF mode can be directly derived, which is stated in Theorem 2. If $\epsilon_1, \epsilon_2 = \mathcal{O}(2^{-n})$, then the CCA security bound of the THF is up to $\mathcal{O}(2^{3/4n})$.

Theorem 2. Let $0 \leq \epsilon_1, \epsilon_2 \leq 1$, $h_1, h_2 \leftarrow_{\$} \mathcal{H}'$, $h \leftarrow_{\$} \mathcal{H}$ and $\pi_2, \pi_3 \leftarrow_{\$} \text{Perm}(n)$. If $\mathcal{H}'$ is an ϵ_1 -AXUHF, and $\mathcal{H}$ is an ϵ_2 -AUHF. Besides, $h_1(\mathbf{t}) \oplus \pi_1(\mathbf{m})$, $h_2(\mathbf{t}) \oplus \pi_4^{-1}(\mathbf{m})$ and h are pairwise independent. Then, for $q \leq 2^{n-2}$, we have

$$\text{Adv}_{\text{THF}}^{\text{ind-cca}}(q) \leq \epsilon(q, n),$$

where $\epsilon(q, n) = 2q^2\epsilon_1^{1.5} + 2^{2-n}q^4\epsilon_1^2 + 2^{5-2n}q^4\epsilon_1 + 13 \cdot 2^{-3n}q^4 + q^2\epsilon_1^2 + q^2\epsilon_1\epsilon_2 + 2^{1-2n}q^2$.

Related-cipher Attacks. Different tweak lengths correspond to different cipher instances. For different tweak input lengths, some hash functions may have trivial collisions (see [53]). These collisions could lead to vulnerabilities between different cipher instances. To mitigate potential security risks, one can incorporate the tweak length into the design of the hash function. However, in this paper, we do not make any claims regarding the attack security between these different cipher instances.

4 Rationale for THF Mode

In practice, symmetric cipher designers often follow a “*prove-then-prune*” methodology: they begin with constructions that offer strong provable security guarantees and then incrementally optimize for performance, provided that the essential security properties remain intact.

In this section, we assess the practical security of tweakable block cipher modes under the *general practical cryptanalysis* framework [27], with a particular focus on resistance to multiple-tweak attacks.

We focus our analysis on three constructions: 4-LRW1, 2-LRW2, and THF. The first two are well-studied and widely regarded as the minimal-permutation instantiations of the CLRW1 and CLRW2 paradigms, respectively, that still achieve BBB CCA-security in the idealized model. Moreover, both share structural affinities with the THF design, making them natural benchmarks for comparison.

Specifically, the THF mode inherits the structural blueprint of 4-LRW1 but replaces identity-based tweak processing with hash-based mechanisms. Additionally, the first and fourth permutations in THF are not required to be random. Compared to 2-LRW2, THF introduces precisely two additional outer permutations corresponding to the aforementioned first and fourth permutations.

Our evaluation will enable an intuitive and fair comparison of the security margins attainable when constructing tweakable block ciphers from standard block ciphers via these modes. In addition, we will provide a latency analysis under comparable security assumptions, highlighting the THF mode’s greater potential for low-latency tweakable block cipher design.

Settings. Let $\mathbf{m} \in \{0, 1\}^n$ and $\mathbf{t} \in \{0, 1\}^\tau$ be the message and tweak, respectively. The encryption functions of the three constructions are defined as follows:

$$E_{\text{THF}} = R^d \left(R^c \left(R^b \left(R^a(\mathbf{m}) \oplus h_1(\mathbf{t}) \right) \oplus h(\mathbf{t}) \right) \oplus h_2(\mathbf{t}) \right),$$

$$E_{4\text{-LRW1}} = R^{\tilde{d}} \left(R^{\tilde{c}} \left(R^{\tilde{b}} \left(R^{\tilde{a}}(\mathbf{m}) \oplus \mathbf{t} \right) \oplus \mathbf{t} \right) \oplus \mathbf{t} \right),$$

$$E_{2\text{-LRW2}} = R^{\hat{c}} \left(R^{\hat{b}}(\mathbf{m} \oplus h_1(\mathbf{t})) \oplus h(\mathbf{t}) \right) \oplus h_2(\mathbf{t}),$$

where $R^i(\cdot)$ denotes the application of i rounds of a keyed round function R , $\tau = n$ for $E_{4\text{-LRW1}}$, and $h = h_1 \oplus h_2$. Moreover, all parameters $b, \dots, \hat{c}$, except for a and d , are strictly positive, and $a, d \geq 0$.

We assume that h_1 and h_2 are independently drawn from a 2^{-n} -AXUHF, with equal latency. The block size is assumed to satisfy $n \geq 64$, as required by our design.

The Latency Evaluation. Under this assumption, and by neglecting the latency introduced by a small number of XOR operations, the total latency (denoted by $T(\cdot)$) of each construction can be estimated as follows:

$$T(E_{\text{THF}}) = \max(T(h_1), a \cdot T(R)) + (b + c + d) \cdot T(R),$$

$$T(E_{4\text{-LRW1}}) = (\tilde{a} + \tilde{b} + \tilde{c} + \tilde{d}) \cdot T(R),$$

$$T(E_{2\text{-LRW2}}) = T(h_1) + (\hat{b} + \hat{c}) \cdot T(R).$$

Remark. In the following analysis, since not all expressions allow for direct quantitative comparison, we adopt a design-oriented perspective to derive general judgments. Specifically, we intentionally disregard highly exceptional or degenerate cases and focus on typical, broadly applicable conclusions. For instance, we may assume that instances with more rounds generally offer stronger security, without delving into edge cases where specific properties of the round function may lead to unexpected degradation.

4.1 The Security of Constructions

To maintain the security of the constructions, note that certain components are required to behave as random permutations. Consequently, their instantiations R^i must satisfy a minimum number of rounds to prevent structural collision-based attacks. Since we will analyze major practical attacks in later sections, we do not assume that these R^i 's behave as full random permutations. Instead, we adopt an aggressive (pruning-based) approach and aim to relax the rounds requirements as much as possible, subject to the constraint that the adversary cannot mount a construction-based collision attack by recovering some R^i through key guessing within the security bounds. Let us denote the minimal number of rounds required to ensure this condition by r_C . Then we assume:

$$b, c, \tilde{a}, \tilde{b}, \tilde{c}, \tilde{d}, \hat{b}, \hat{c} \geq r_C.$$

4.2 Single-tweak attacks

The single-tweak (ST) security of each construction reduces to the security of the underlying round functions:

$$E_{\text{THF}}, E_{4\text{-LRW1}}, E_{2\text{-LRW2}} \iff R^{a+b+c+d}, R^{\tilde{a}+\tilde{b}+\tilde{c}+\tilde{d}}, R^{\hat{b}+\hat{c}}, \text{ respectively.}$$

Assuming that the round function R requires at least r_S rounds to meet the desired security level, we obtain the following lower bounds:

$$a + b + c + d \geq r_S, \quad \tilde{a} + \tilde{b} + \tilde{c} + \tilde{d} \geq r_S, \quad \hat{b} + \hat{c} \geq r_S.$$

4.3 Multiple-tweak attacks

We mainly consider three highly threatening categories of multiple-tweak (MT) attacks: MT differential attacks, MT linear attacks, and MT algebraic attacks.

MT differential attacks. Given differences $\delta_m, \delta_t, \delta_c \in \{0, 1\}^n$ with at least two of them nonzero, we analyze the differential propagation

$$(\delta_m, \delta_t) \xrightarrow{E} \delta_c.$$

- For $E = E_{\text{THF}}$, all trails follow a unified structure determined by the choice of $\delta_1, \delta_2, \delta_3, \gamma_1, \gamma_2$, and the probability of each trail is given by

$$\begin{aligned} p_{\text{THF}} &= \Pr(\delta_m \xrightarrow{R^a} \delta_1) \cdot \Pr(\delta_1 \oplus \gamma_1 \xrightarrow{R^b} \delta_2) \cdot \\ &\quad \Pr(\delta_2 \oplus \gamma_1 \oplus \gamma_2 \xrightarrow{R^c} \delta_3) \cdot \Pr(\delta_3 \oplus \gamma_2 \xrightarrow{R^d} \delta_c) \cdot \\ &\quad \Pr(\delta_t \xrightarrow{h_1} \gamma_1) \cdot \Pr(\delta_t \xrightarrow{h_2} \gamma_2). \end{aligned}$$

We first consider the trivial case where $\delta_t = \mathbf{0}$. In this case, by setting $\gamma_1 = \gamma_2 = \mathbf{0}$, the probability p_{THF} coincides with that in the ST setting. We now focus on the non-trivial case where $\delta_t \neq \mathbf{0}$. Since h_1 and h_2 are independently sampled from a 2^{-n} -AXUHF, for any $\gamma_1, \gamma_2 \in \{0, 1\}^n$, it holds that

$$\Pr(\delta_t \xrightarrow{h_1} \gamma_1) = \Pr(\delta_t \xrightarrow{h_2} \gamma_2) = 2^{-n}.$$

Consequently, the differential probability p_{THF} is upper bounded by 2^{-2n} . Furthermore, when $n \geq 64$, it becomes practically infeasible to construct more than 2^n valid differential trails of the form

$$(\delta_m, \delta_t) \xrightarrow{E} \delta_c$$

such that their cumulative probability exceeds 2^{-n} , which is typically required for a successful differential attack.

Therefore, we conclude that when the block size satisfies $n \geq 64$, E_{THF} is secure against the MT differential analysis.

- For $E = E_{4\text{-LRW1}}$, all trails are characterized by the sequence $\delta_1, \delta_2, \delta_3$, and the probability of each trail is

$$\begin{aligned} p_{4\text{-LRW1}} &= \Pr(\delta_m \xrightarrow{R^{\bar{a}}} \delta_1) \cdot \Pr(\delta_1 \oplus \delta_t \xrightarrow{R^{\bar{b}}} \delta_2) \cdot \\ &\quad \Pr(\delta_2 \oplus \delta_t \xrightarrow{R^{\bar{c}}} \delta_3) \cdot \Pr(\delta_3 \oplus \delta_t \xrightarrow{R^{\bar{d}}} \delta_c). \end{aligned}$$

Here, the tweak difference δ_t is actively involved in each component of the cipher and can be deliberately chosen to maximize the transition probability. Consequently, for $\delta_m, \delta_c \neq \mathbf{0}$, the maximum differential probability over all choices of δ_t satisfies

$$\max_{\delta_t} p_{4\text{-LRW1}} \geq \Pr(\delta_m \xrightarrow{R^{\bar{a}}} \delta_1) \cdot \Pr(\delta_1 \xrightarrow{R^{\bar{b}}} \delta_2) \cdot \Pr(\delta_2 \xrightarrow{R^{\bar{c}}} \delta_3) \cdot \Pr(\delta_3 \xrightarrow{R^{\bar{d}}} \delta_c).$$

This indicates that, in the MT setting, the maximum probability of differential trails is always at least as large as in the ST case. In addition, the following two cases can also potentially lead to differential trails with higher probability. For the case $\delta_m = \mathbf{0}$, by setting $\delta_1 = \mathbf{0}$, the trail probability simplifies to

$$p_{4\text{-LRW1}} = \Pr(\delta_t \xrightarrow{R^{\bar{b}}} \delta_2) \cdot \Pr(\delta_2 \oplus \delta_t \xrightarrow{R^{\bar{c}}} \delta_3) \cdot \Pr(\delta_3 \oplus \delta_t \xrightarrow{R^{\bar{d}}} \delta_c).$$

Similarly, when $\delta_c = \mathbf{0}$, setting $\delta_3 = \delta_t$ yields

$$p_{4\text{-LRW1}} = \Pr(\delta_m \xrightarrow{R^{\bar{a}}} \delta_1) \cdot \Pr(\delta_1 \oplus \delta_t \xrightarrow{R^{\bar{b}}} \delta_2) \cdot \Pr(\delta_2 \oplus \delta_t \xrightarrow{R^{\bar{c}}} \delta_t).$$

These observations suggest that the resistance of $E_{4\text{-LRW1}}$ against MT differential attacks is typically weaker than that against ST differential attacks.

- For $E = E_{2\text{-LRW2}}$, the differential trails are defined by $\delta_1, \delta_2, \gamma_1, \gamma_2$, and the probability of each trail is

$$\begin{aligned} p_{2\text{-LRW2}} &= \Pr(\delta_m \oplus \gamma_1 \xrightarrow{R^{\bar{b}}} \delta_2) \cdot \Pr(\delta_2 \oplus \gamma_1 \oplus \gamma_2 \xrightarrow{R^{\bar{c}}} \delta_c \oplus \gamma_2) \cdot \\ &\quad \Pr(\delta_t \xrightarrow{h_1} \gamma_1) \cdot \Pr(\delta_t \xrightarrow{h_2} \gamma_2). \end{aligned}$$

Similarly to the analysis of E_{THF} , we conclude that $E_{2\text{-LRW2}}$ achieves resistance against MT differential attacks, provided that $n \geq 64$.

MT Linear Attacks. Given masks $\lambda_m, \lambda_t, \lambda_c \in \{0, 1\}^n$ with at least two of them nonzero, we consider the linear propagation:

$$(\lambda_m, \lambda_t) \xrightarrow{E} \lambda_c.$$

- For $E = E_{\text{THF}}$, all linear trails can be characterized by a fixed selection of intermediate masks $\lambda_1, \lambda_2, \lambda_3$, and the correlation of each trail is given by:

$$\begin{aligned} c_{\text{THF}} &= \text{Cor}(\lambda_m \xrightarrow{R^a} \lambda_1) \cdot \text{Cor}(\lambda_1 \xrightarrow{R^b} \lambda_2) \cdot \text{Cor}(\lambda_2 \xrightarrow{R^c} \lambda_3) \\ &\quad \cdot \text{Cor}(\lambda_3 \xrightarrow{R^d} \lambda_c) \cdot \text{Cor}\left(\lambda_t \xrightarrow{h_1, h_2, h_1 \oplus h_2} \lambda_1, \lambda_3, \lambda_2\right). \end{aligned} \tag{6}$$

We define the tweak-related correlation term as

$$\text{Cor}\left(\lambda_t \xrightarrow{h_1, h_2, h_1 \oplus h_2} \lambda_1, \lambda_3, \lambda_2\right) \triangleq c_h, \tag{7}$$

which satisfies, by definition,

$$c_h = 2 \Pr\left(\lambda_t^T \cdot t \oplus (\lambda_1 \oplus \lambda_2)^T \cdot h_1 \oplus (\lambda_3 \oplus \lambda_2)^T \cdot h_2 = 0\right) - 1.$$

If the tweak t is fixed, then $c_h = 1$, which is equivalent to the ST scenario. When the tweak varies, and both plaintext and ciphertext are also allowed to vary, the overall trail correlation c_{THF} cannot exceed that in the ST case.

In particular, if either the plaintext or ciphertext is fixed, the correlation simplifies to:

$$c_{\text{THF}} = \text{Cor}(\lambda_1 \xrightarrow{R^b} \lambda_2) \cdot \text{Cor}(\lambda_2 \xrightarrow{R^c} \lambda_3) \cdot \text{Cor}(\lambda_3 \xrightarrow{R^d} \lambda_c) \cdot c_h,$$

or

$$c_{\text{THF}} = \text{Cor}(\lambda_m \xrightarrow{R^a} \lambda_1) \cdot \text{Cor}(\lambda_1 \xrightarrow{R^b} \lambda_2) \cdot \text{Cor}(\lambda_2 \xrightarrow{R^c} \lambda_3) \cdot c_h.$$

These observations highlight that the linear security of THF in the MT setting also depends on the value of c_h . A smaller c_h leads to a lower overall correlation, thereby enhancing the cipher's resistance to MT linear attacks.

- For $E = E_{4\text{-LRW1}}$, all trails are characterized by the choice of λ_1, λ_2 , and the trail correlation is:

$$\begin{aligned} c_{4\text{-LRW1}} &= \text{Cor}(\lambda_m \xrightarrow{R^a} \lambda_1) \cdot \text{Cor}(\lambda_1 \xrightarrow{R^b} \lambda_2) \\ &\quad \cdot \text{Cor}(\lambda_2 \xrightarrow{R^c} \lambda_1 \oplus \lambda_2 \oplus \lambda_t) \cdot \text{Cor}(\lambda_1 \oplus \lambda_2 \oplus \lambda_t \xrightarrow{R^d} \lambda_c). \end{aligned} \quad (8)$$

This matches the form under the ST setting. However, when the plaintext or ciphertext is fixed, i.e., $\text{Cor}(\lambda_m \xrightarrow{R^a} \lambda_1) = 1$ or $\text{Cor}(\lambda_1 \oplus \lambda_2 \oplus \lambda_t \xrightarrow{R^d} \lambda_c) = 1$, the resulting correlation $c_{4\text{-LRW1}}$ becomes larger, weakening the linear security compared to the ST case.

- For $E = E_{2\text{-LRW2}}$, the trails involve intermediate masks λ_2 , and the corresponding trail correlation is:

$$\begin{aligned} c_{2\text{-LRW2}} &= \text{Cor}(\lambda_m \xrightarrow{R^b} \lambda_2) \cdot \text{Cor}(\lambda_2 \xrightarrow{R^c} \lambda_c) \\ &\quad \cdot \text{Cor}\left(\lambda_t \xrightarrow{h_1, h_2, h_1 \oplus h_2} \lambda_m, \lambda_c, \lambda_2\right). \end{aligned}$$

Since $c_h \leq 1$ by definition, the ST linear security of $E_{2\text{-LRW2}}$ directly implies its security against MT linear attacks.

MT Algebraic Attacks. For non-zero $\mathbf{u}$, we focus on the algebraic degree of the ciphertext $\mathbf{c}$ or the plaintext $\mathbf{m}$ with respect to $\mathbf{t}_u$, i.e., $\deg(E, \mathbf{u})$ and $\deg(E^{-1}, \mathbf{u})$, respectively.

- For $E = E_{\text{THF}}$, the longest algebraic degree trail during encryption follows the path $\mathbf{t} \rightarrow h_1 \rightarrow R^{b+c+d}$, while during decryption it follows $\mathbf{t} \rightarrow h_2 \rightarrow R^{-(a+b+c)}$. Therefore, the algebraic degree can be approximated as:

$$\begin{aligned} \deg(E_{\text{THF}}, \mathbf{u}) &\approx \deg(R^{b+c+d} \circ h_1, \mathbf{u}), \\ \deg(E_{\text{THF}}^{-1}, \mathbf{u}) &\approx \deg(R^{-(a+b+c)} \circ h_2, \mathbf{u}). \end{aligned} \quad (9)$$

This indicates that the resistance to algebraic attacks in the MT setting also depends on the algebraic degree of the hash functions h_1 and h_2 .

- For $E = E_{4\text{-LRW1}}$, we have:

$$\begin{aligned}\deg(E_{4\text{-LRW1}}, \mathbf{u}) &\approx \deg(R^{\tilde{b}+\tilde{c}+\tilde{d}}, \mathbf{u}), \\ \deg(E_{4\text{-LRW1}}^{-1}, \mathbf{u}) &\approx \deg(R^{-(\tilde{a}+\tilde{b}+\tilde{c})}, \mathbf{u}).\end{aligned}\tag{10}$$

The tweak-dependent algebraic degree in both forward and inverse directions is reduced in the MT setting. This results in weaker resistance against algebraic attacks compared to the ST scenario.

- For $E = E_{2\text{-LRW2}}$, we have the following approximations:

$$\begin{aligned}\deg(E_{2\text{-LRW2}}, \mathbf{u}) &\approx \deg(R^{\hat{b}+\hat{c}} \circ h_1, \mathbf{u}), \\ \deg(E_{2\text{-LRW2}}^{-1}, \mathbf{u}) &\approx \deg(R^{-(\hat{b}+\hat{c})} \circ h_2, \mathbf{u}).\end{aligned}$$

These expressions indicate that the algebraic degree of the cipher under MT scenarios is not lower than that under ST scenarios. Therefore, the ST algebraic security of $E_{2\text{-LRW2}}$ inherently implies its security against algebraic attacks in the MT setting.

4.4 Comparison and Summary

We summarize the analysis results in Table 1. Rows 2–4 compare the security of the MT setting against the corresponding ST setting across three types of cryptanalytic evaluations. Row 5 presents the minimum number of rounds required to achieve the desired security level under both the ST and MT settings. Rows 6 and 7 report the encryption and decryption latencies of the secure constructions.

From the analysis, it is observed that the E_{THF} and $E_{4\text{-LRW1}}$ generally require larger values of parameters $b, c, \tilde{b}, \tilde{c}$ to achieve higher resistance to MT attacks. In contrast, for $E_{2\text{-LRW2}}$, larger values of $\hat{b}, \hat{c}$ are required to maintain security against ST attacks. As a result, we assume that the parameters $b, c, \tilde{b}, \tilde{c}, \hat{b}, \hat{c} \geq r_C$ can typically be satisfied. Based on these assumptions, we derive the encryption and decryption latencies for three ciphers, as shown in the table.

For E_{THF} , assuming that $a + b + c + d = r_S$, we begin with $a = d = 0$ and increment the values of a and d . There is likely a critical threshold where increasing a and d will still maintain resistance against both linear and algebraic attacks. However, beyond this threshold, further increases in a and d will make the construction insecure. This is because when $a = d = 0$, the construction effectively becomes equivalent to $E_{2\text{-LRW2}}$. From the table, we can observe that $E_{2\text{-LRW2}}$ already has redundancy in its resistance to MT linear and algebraic attacks. Therefore, the latency of E_{THF} has a greater potential for lower latency compared to $E_{2\text{-LRW2}}$.

The presence of hash functions in E_{THF} enhances its security relative to $E_{4\text{-LRW1}}$ under the same rounds distribution. This can be confirmed through Equations 6, 8, 9 and 10. As a result, for the same level of security, E_{THF} generally requires fewer rounds. Moreover, if the hash functions h_1 and h_2 are designed to have low-latency characteristics, especially if their latency is less than or

Metric	E_{THF}	$E_{4\text{-LRW1}}$	$E_{2\text{-LRW2}}$
MT differential analysis	–	↓	–
MT linear analysis	D	↓	↑
MT algebraic attack	D	↓	↑
#Rounds requirement	$a+b+c+d \geq r_S$ $b, c \geq r_C$	$\tilde{a}+\tilde{b}+\tilde{c}+\tilde{d} > r_S$ $\tilde{a}, \tilde{b}, \tilde{c}, \tilde{d} \geq r_C$	$\hat{b}+\hat{c} = r_S$ $\hat{b}, \hat{c} \geq r_C$
Encryption latency	$\geq r_S \cdot T(R) + \Delta_a$	$> r_S \cdot T(R)$	$r_S \cdot T(R) + T(h_1)$
Decryption latency	$\geq r_S \cdot T(R) + \Delta_d$	$> r_S \cdot T(R)$	$r_S \cdot T(R) + T(h_2)$

Legend: ↓: MT security is typically weaker than ST; ↑: stronger; –: secure in this setting; D: the security depends on the property of hash functions.

$$\Delta_a = \max\{0, T(h_1) - a \cdot T(R)\}, \quad \Delta_d = \max\{0, T(h_2) - d \cdot T(R)\}.$$

Table 1. Comparison of three constructions across security and performance aspects.

equal to $r_C \cdot T(R)$, then E_{THF} exhibits improved latency performance. Thus, the key to constructing low-latency ciphers using the E_{THF} structure lies in selecting a low-latency 2^{-n} -AXUHF, which plays a crucial role in optimizing the overall performance of the cipher. Furthermore, we have identified a low-latency hash functions family, which will be detailed in a later section. This makes it practically feasible to construct low-latency cipher using the THF mode.

5 Instantiation: Blink

In this section, we instantiate the THF mode by proposing a concrete tweakable block cipher family named **Blink**. The **Blink** family supports multiple configurations, including block sizes of 64 and 128 bits, and tweak lengths equal to one or two times the block size. For simplicity, in the following discussion we denote the block size by n and assume that the length of tweak t is fixed to τ .

5.1 The Construction

Blink's overall structure is illustrated in Figure 2. While based on the THF construction, **Blink** introduces several modifications compared to the instantiation E_{THF} in Section 4, to better support hardware efficiency. In particular, **Blink** adopts a reflector construction [20], which reduces the hardware footprint by exploiting structural symmetry. We next explain the notations used in Figure 2.

Each round function consists of five operations: an S-box layer (**S**), a Mix-Column layer (**M**), round key addition (**AK**), round constant addition (**AC**), and a shuffle layer (**P**). The round function is:

$$R = P \circ AC \circ AK \circ M \circ S.$$

We denote $\text{MK}_{\mathbf{k}}(\mathbf{x}) = \text{M}(\mathbf{x}) \oplus \mathbf{k}$, and $\hat{\mathbf{z}} = \text{M}(\mathbf{z})$. Inverses are marked with overlines, e.g., $\bar{P}$. Since **S** and **M** are involutive, the inverse of round function is thus:

$$\bar{R} = S \circ \text{MK}_{\mathbf{r}\mathbf{k} \oplus \mathbf{r}\mathbf{c}} \circ \bar{P}.$$

This shows that the structure depicted in Figure 2 supports a reflection property.

Blink corresponds to the four permutations in THF as:

- $\pi_1: M \circ S \circ R^a(\bullet \oplus w_1)$,
- $\pi_2: M \circ S \circ R^b \circ P$,
- $\pi_3: \bar{P} \circ \bar{R}^b \circ S$,
- $\pi_4: \bar{R}^a \circ S \circ M(\bullet) \oplus w_2$.

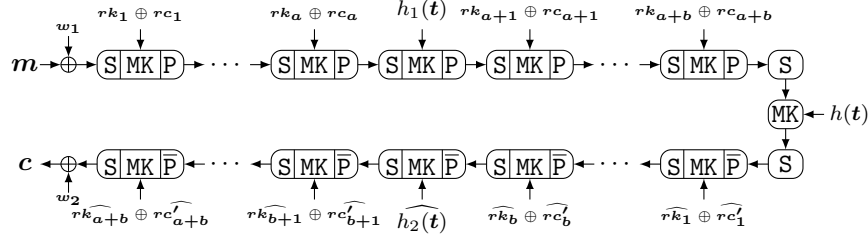


Fig. 2. The overview of Blink

5.2 The Round Function

The internal state of Blink is organized as a two-dimensional array, consisting of 4 rows and $n/16$ columns, where each cell represents a nibble (4 bits). The state $s_{n/4-1} \parallel \dots \parallel s_1 \parallel s_0$ can be visualized as follows:

$$\begin{bmatrix} s_0 & s_1 & \dots & s_{n/16-1} \\ s_{n/16} & s_{n/16+1} & \dots & s_{n/8-1} \\ s_{n/8} & s_{n/8+1} & \dots & s_{3n/16-1} \\ s_{3n/16} & s_{3n/16+1} & \dots & s_{n/4-1} \end{bmatrix},$$

Each operation in round function R updates internal states as follows.

- S: The 4-bit S-box S is applied to each cell in parallel. Its specification in hexadecimal is shown in the following table. Additionally, the S-box is an involution, meaning that its inverse is equal to itself.

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$S(x)$	1	0	9	3	8	5	e	7	4	2	c	b	a	f	6	d

- M: A diffusion matrix M is multiplied to each column. Namely,

$$[s_j, s_{j+n/16}, s_{j+n/8}, s_{j+3n/16}]^T \leftarrow M [s_j, s_{j+n/16}, s_{j+n/8}, s_{j+3n/16}]^T,$$

where $0 \leq j < n/16$, and M is the involutory matrix in Midori:

$$M = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}.$$

- **AK:** The round key is XORed to the state.
- **AC:** The round constant is XORed to the state. The specific values for the round constants $\mathbf{rc}_i$ and $\mathbf{rc}'_i$ are provided in Appendix D.
- **P:** Each cell in the state is shuffled according to the permutation P . Namely,

$$[\mathbf{s}_0, \mathbf{s}_1, \dots, \mathbf{s}_{n/4-1}] \leftarrow [\mathbf{s}_{P[0]}, \mathbf{s}_{P[1]}, \dots, \mathbf{s}_{P[n/4-1]}].$$

Specifically, for 64-bit block

$$P = [0, 5, 11, 10, 1, 6, 4, 13, 2, 12, 9, 15, 3, 7, 14, 8],$$

and for 128-bit block,

$$P = [5, 12, 4, 1, 17, 9, 10, 16, 28, 14, 21, 22, 11, 27, 8, 13, \\ 2, 25, 18, 3, 30, 6, 19, 20, 0, 23, 24, 31, 7, 15, 29, 26]$$

5.3 The Hash Function

We define h_T as $h_T(\mathbf{t}) = T \cdot \mathbf{t}$, where T is an $n \times \tau$ matrix. The hash family based on Toeplitz matrices is then defined as:

$$\mathcal{H} = \{h_T \mid T \text{ is an keyed } n \times \tau \text{ Toeplitz matrix}\}.$$

A Toeplitz matrix is a matrix in which each element on the same diagonal is identical, meaning it is uniquely determined by the values of $n + \tau - 1$ diagonals. Specifically, the Toeplitz matrix T with $\mathbf{k} = (k_0, k_1, \dots, k_{n+\tau-2})$ is defined as

$$T = \begin{bmatrix} k_{n-1} & k_n & \cdots & k_{n+\tau-2} \\ k_{n-2} & k_{n-1} & \cdots & k_{n+\tau-3} \\ \vdots & \vdots & \ddots & \vdots \\ k_0 & k_1 & \cdots & k_{\tau-1} \end{bmatrix}. \quad (11)$$

In the **Blink**, h_1 and h_2 are randomly selected from the hash family $\mathcal{H}$, and they are independently determined by the $(n + \tau - 1)$ -bit keys $\mathbf{k}_1$ and $\mathbf{k}_2$, respectively. The function h is defined as the sum of h_1 and h_2 , i.e.,

$$h(\mathbf{t}) = h_1(\mathbf{t}) \oplus h_2(\mathbf{t}).$$

Table 2. Security claims and parameter choices.

Version	n	τ	a	b	Key Size	Time	Data
Blink-64a	64	64	2	3	448	2^{128}	2^{56}
Blink-64b	64	128	2	3	448	2^{128}	2^{56}
Blink-128a	128	128	3	3	1024	2^{128}	2^{80}
Blink-128b	128	256	3	3	1024	2^{128}	2^{80}
Blink-128A	128	128	3	5	1280	2^{256}	2^{80}
Blink-128B	128	256	3	5	1280	2^{256}	2^{80}

5.4 The Key Schedule

The master key $\mathbf{k}$ is the concatenation of whitening keys $\mathbf{w}_1$, $\mathbf{w}_2$, and round keys $\mathbf{rk}_1, \dots, \mathbf{rk}_{a+b}$:

$$\mathbf{k} = \mathbf{rk}_{a+b} \parallel \dots \parallel \mathbf{rk}_1 \parallel \mathbf{w}_2 \parallel \mathbf{w}_1,$$

with a total length of $(a + b + 2)n$ bits.

Let $\mathbf{k}'$ be a rearrangement of $\mathbf{k}$, where each bit $k'_i = k_{11 \cdot i \bmod (a+b+2)n}$. The key $\mathbf{k}_2 \parallel \mathbf{k}_1$, used for generating the hash functions h_1 and h_2 , is derived from the least significant $2n + 2\tau - 2$ bits of $\mathbf{k}'$.

5.5 Security Claims and Parameter Choices

According to different security requirements, the number of rounds for each version is summarized in Table 2. Similarly to QARMAv2 [7], we restrict the data volume per master key to at most 2^{56} blocks for the 64-bit variant and 2^{80} blocks for the 128-bit variant. This setting is consistent with existing recommendations, such as those from the NIST call for lightweight cryptographic algorithms. We also believe that these limits are ample for real-world applications.

Remark. We do not claim the security of Blink against related-key attacks.

6 Design Rationale for Blink

This section outlines the design rationale behind the Blink family. Its primary goal is to instantiate the THF construction to demonstrate its low-latency potential, with a particular focus on memory encryption scenarios.

6.1 Design of the Hash Function

As described in Section 4.4, the key to constructing a low-latency tweakable block cipher via the THF mode is the identification of an efficient AXUHF. Universal hash functions have a wide range of applications, including message authentication codes [59, 19, 24], authenticated encryption schemes [48], and tweakable block ciphers [41, 36, 45]. Most constructions rely on basic mathematical operations,

such as modular multiplication, polynomial multiplication, matrix multiplication, or finite field multiplication [23,47,54,39,19,1,50,28]. Recently, AES-based hash functions have been proposed that leverage hardware acceleration to improve efficiency [12].

We observe that matrix-based universal hash functions exhibit superior latency characteristics. In particular, the family

$$\mathcal{H}_3 = \{h_A \mid A \text{ is a keyed } n \times \tau \text{ matrix}\}$$

can be implemented with latency of only $\lceil \log_2(\tau) \rceil \cdot T(\text{XOR/NXOR}) + T(\text{NAND})$, yielding a circuit depth (as defined in Definition 1) of $2\lceil \log_2(\tau) \rceil + 1$. This construction achieves 2^{-n} -almost XOR universality, with key size $n \cdot \tau$ bits.

Based on this, [47] showed that constraining A to be a Toeplitz matrix preserves the 2^{-n} -AXU property while reducing the key size to $n + \tau - 1$ bits. Another construction, proposed by [39], uses an n -bit linear feedback shift register (LFSR) to hash messages, but this limits the key size to n and requires the initialization to be the coefficients of an irreducible polynomial of degree n . Given that a 256-bit tweak is sufficient for most applications, we adopt the Toeplitz matrix-based construction due to its efficient key size and relaxed structural constraints, which simplify key generation.

Recall that the hash family based on Toeplitz matrices is:

$$\mathcal{H} = \{h_T \mid T \text{ is a keyed } n \times \tau \text{ Toeplitz matrix}\},$$

with the structure of the matrix given in Equation 11.

The circuit depth of any $h_T \in \mathcal{H}$ is equal to that of $h_A \in \mathcal{H}_3$, namely,

$$\text{depth}(h_T) = 2\lceil \log_2(\tau) \rceil + 1.$$

For typical values of τ , such as 64, 128, and 256, the corresponding depths are 13, 15, and 17, respectively. This low latency makes the Toeplitz-based hash function particularly suitable for embedding within the THF mode to construct low-latency tweakable block ciphers.

In addition to its 2^{-n} -AXU property, we examine the correlation c_h (as defined in Equation 7) and the algebraic degree to evaluate the resistance offered by h_T for E_{THF} against MT linear and algebraic attacks.

For variable tweaks and $\lambda_t \neq \mathbf{0}$, we must have $\lambda_1 \oplus \lambda_2 \neq \mathbf{0}$ or $\lambda_3 \oplus \lambda_2 \neq \mathbf{0}$. The linear approximation holds with probability 1 with probability 2^{-n} , and $1/2$ otherwise. Consequently, the expected correlation satisfies:

$$c_h = 2 \left(2^{-n} + \frac{1 - 2^{-n}}{2} \right) - 1 = 2^{-n},$$

offering exponential resistance to MT linear attacks.

Furthermore, since h_T is linear in the tweak input t , its algebraic degree in the variable $\mathbf{t}_u$ is 1, indicating negligible contribution to algebraic attacks. Thus, h_T is well suited for extending low-latency block ciphers (where algebraic attacks are not a primary threat) into tweakable ones with minimal latency overhead.

6.2 Design of Round Function

Blink is designed for memory encryption, where the read/write unit corresponds to a cache line, typically 512 or 1024 bits. Unlike block ciphers used in modes like CTR, where only encryption is invoked, tweakable block ciphers are typically required to support both encryption and decryption. Therefore, to align with typical cache line boundaries, the block size in **Blink** is restricted to commonly used values as 64 or 128 bits.

Among existing low-latency ciphers with these block sizes, **Midori** [8] demonstrates strong resistance against differential and linear attacks. Its round function has been adopted in **Mantis** [14], and its MixColumn layer reused in the permutation **Orthros** [9]. Building on this, **Blink** designs its round function based on the structure of **Midori**.

As **Blink** adopts a reflection structure, minimizing the latency of the round function R and its inverse $\bar{R}$ is essential for overall performance. Each component of the round function is thus optimized for both forward and inverse computations, ensuring low latency in both directions.

S-boxes. Since **Blink** uses 64- or 128-bit blocks, we consider 4- or 8-bit S-boxes accordingly. For 8-bit low-latency S-boxes proposed in the literature [51], the best cryptographic properties achieved are linearity 128 and uniformity 64. However, these properties do not demonstrate an improvement over 4-bit S-boxes in terms of cryptographic strength. By opting for 4-bit S-boxes, the number of active S-boxes can propagate over a larger space, offering stronger resistance to differential and linear attacks. This advantage has also been highlighted in the design of **QARMAv2** [7].

Therefore, we adopt a 4-bit S-box for both the 64-bit and 128-bit variants of **Blink**, guided by the following security criteria: (a) bijectivity; (b) differential uniformity of 4; (c) linearity of 8; (d) full diffusion, i.e., each input bit influences all output bits.

To minimize latency, we target an S-box and its inverse with a depth of at most 3.5. Since involutory S-boxes are their own inverses, their forward and inverse depths are identical. Thus, we focus on identifying involutory S-boxes that satisfy the above criteria, simplifying the search process and ensuring low-latency implementation in both encryption and decryption.

In the study by Rasoolzadeh [51], three *extended bit permutation equivalence* classes (as defined in Appendix A) of 4-bit S-boxes were identified that meet the following criteria: involutory property, differential uniformity of 4, linearity of 8, and a depth of 3.5⁵. Among these, only the first equivalence class guarantees full diffusion. Notably, each output bit of the representative S-box in this class achieves the maximum algebraic degree of 3 (see Appendix A), effectively compensating for the low algebraic degree of the Toeplitz matrix-based hash function. Therefore, an involutory S-box from this class is selected for **Blink**. Its

⁵ The depth corresponds to a latency complexity of 3, as defined in [51].

implementation is shown below, where $\overline{x_i}$ denotes $\text{NOT}(x_i)$.

$$\begin{aligned} y_0 &= ((\overline{x_2} \text{ NAND } \overline{x_1}) \text{ NOR } \overline{x_0}) \text{ NOR } ((x_3 \text{ NOR } x_2) \text{ NOR } x_0) \\ y_1 &= ((\overline{x_2} \text{ NAND } \overline{x_0}) \text{ NAND } (\overline{x_3} \text{ NAND } \overline{x_1})) \text{ NOR } ((x_3 \text{ NAND } x_0) \text{ NOR } (x_2 \text{ NAND } x_1)) \\ y_2 &= ((\overline{x_1} \text{ NAND } \overline{x_0}) \text{ NAND } x_2) \text{ NAND } ((x_2 \text{ NOR } x_0) \text{ NAND } x_3) \\ y_3 &= ((\overline{x_0} \text{ NAND } x_2) \text{ NOR } (x_3 \text{ NAND } x_1)) \text{ NOR } ((\overline{x_3} \text{ NAND } x_0) \text{ NAND } (\overline{x_2} \text{ NAND } \overline{x_1})). \end{aligned}$$

Linear Layer. We adopt 4×4 almost MDS (maximum distance separable) binary matrix M used in Midori [8], which offers significantly lower latency compared to MDS matrices. Additionally, M is an involutory matrix, ensuring identical latency for both forward and inverse computations.

The cell permutation for shuffle layer is primarily designed to follow the wide-trail strategy [26], which requires mapping cells from the same column to distinct columns, as stated in Condition 1. Consequently, after the permutation, each column must consist of cells originating from different input columns.

Condition 1 *For each column $[s_i, s_{i+4}, s_{i+8}, s_{i+12}]$ in the $4 \times 4 \times \frac{n}{64}$ array, after applying the shuffle layer, these cells will be mapped to 4 cells in different columns, where $i \in \{j + 16l \mid j \in \{0, \dots, 3\}, l \in \{0, \dots, \frac{n}{64} - 1\}\}$.*

For the 64-bit block size, the design of the shuffle layer in conjunction with the diffusion matrix M was thoroughly analyzed by Alfarano *et al.* [2]. Based on the classification in Appendix C.1 of [2], we select a permutation from the first class that satisfies Condition 1 and ensures full diffusion within three rounds in both forward and backward directions. Furthermore, this permutation achieves the optimal lower bound on the number of active S-boxes over the first eight rounds.

For the 128-bit block size, we introduce an additional requirement, Condition 2, to further improve diffusion. Together with Condition 1, it ensures that each output cell is influenced by exactly 23 input cells after three rounds, in both forward and backward directions. As shown in Appendix B, increasing the number of contributing input columns per output cell enhances overall diffusion—motivating the introduction of Condition 2. Notably, among all permutations satisfying Condition 1, the maximum attainable minimum number of influencing input cells per output cell is 23.

Condition 2 *For the case $n = 128$, after applying the permutation three times, each output cell should be influenced by input cells from at least 7 distinct columns, and each input cell should contribute to output cells in at least 7 distinct columns.*

Given the vast search space, randomly identifying permutations satisfying both conditions is infeasible. To overcome this, we formulated the conditions as MILP constraints and used the Gurobi solver to systematically explore a set of feasible candidates. For each candidate, we computed the lower bounds on the number of active S-boxes under both ST differential and linear attacks, and selected the permutation with the best bounds.

Round Constants. The round constants rc_i, rc'_i in 128-bit block version are derived from the fractional part of $\pi = 3.14159\dots$, similar to PRINCE [20]. In the 64-bit block version, each round constant is taken as the least significant 64 bits of corresponding the round constant used in the 128-bit block version. Detailed values of these constants can be found in Appendix D.

6.3 Design of Rounds Number

The latency of Toeplitz hash function family is approximately $2\lceil\log_2(\tau)\rceil + 1$. For specific values of τ , such as 64, 128, and 256, the corresponding depths are 13, 15, and 17, respectively. Considering the delay overhead introduced by high fan-out, and given that the depth of a single round function is 7.5, the overall latency of the hash function is roughly equivalent to that of three to four rounds.

As discussed in Section 4, increasing the parameter a improves parallelism and helps hide the latency of the hash function. However, a larger value of a may also increase the total number of rounds required to achieve a sufficient security level. Therefore, we selected parameters a and b by jointly considering both parallelizability and the results of our security analysis, aiming to achieve near-optimal latency.

Notably, due to the strong resistance of **Blink** against ST algebraic attacks, fewer rounds are sufficient to meet the ST security requirement. As a result, we selected a such that the latency of computing $h(t)$ is roughly hidden by the parallel execution of the encryption rounds, resulting in nearly the same total number of rounds required for both ST and MT security.

6.4 Design of Key Schedule

Unlike most symmetric-key encryption algorithms, the master key length in **Blink** exceeds the targeted security level. A longer key eliminates the need for a complex key schedule design and ensures the independence between the sub-keys k_1 and k_2 used for hash generation. Moreover, a long key offers enhanced security compared to a shorter one, which is particularly advantageous when designing low-latency algorithms.

The design for the central construction of **Blink** deliberately avoids adopting the α -reflection property, which is defined as $F_k = F_{k \oplus \alpha}^{-1}$, in order to mitigate attacks associated with this structural property. Both encryption and decryption require the use of rk_i and $\hat{rk}_i$ in a specific order. During encryption, the round keys are used in the following sequence:

$$rk_1, \dots, rk_{a+b}, \hat{rk}_1, \dots, \hat{rk}_{a+b}.$$

For decryption, the order is reversed:

$$rk_{a+b}, \dots, rk_1, \hat{rk}_{a+b}, \dots, \hat{rk}_1.$$

The randomness in the choice of the round constants ensures that $\forall i$,

$$rk_i \oplus rc_i \oplus rk_{a+b-i} \oplus rc'_{a+b-i} \neq rk_{a+b-i} \oplus rc_{a+b-i} \oplus rk_i \oplus rc'_i,$$

Table 3. The lower bounds on the number of active S-boxes for **Blink**

Block Size	Half-Cipher								Full Cipher							
	1	2	3	4	5	6	7	8	2	4	6	8	10	12		
64	1	4	7	16	23	30	35	38	4	16	24	32	50	64		
128	1	4	7	16	24	36	48	58	4	16	24	40	64	-		

preventing the existence of special weak keys that could cause **Blink** to satisfy the α -reflection property.

7 Security Analysis

7.1 Differential Analysis

We employed the MILP approach for differential analysis [18]. The obtained results, including bounds on the active S-boxes numbers, are summarized in Table 3, where "full cipher" refers to the full reflective construction. Each round containing an S-box layer is counted as a single round in the overall round count.

For **Blink-64**, since the maximum differential probability of the S-box in **Blink** is 2^{-2} , the maximum differential probability of 10-round "full cipher" differential characteristic is at most 2^{-100} . Hence, we conclude that when the data does not exceed 2^{56} , it is highly unlikely to distinguish 10 rounds of **Blink-64**.

Furthermore, due to the fact that after applying **R** or its inverses three times, each output cell is influenced by all input cells and any three consecutive rounds keys are independent, the output of the third encryption or decryption round is affected by $(16 + 9 + 3) \times 4 = 112$ bits of the key, while the second encryption or decryption round is affected by $(9 + 3) \times 4 = 48$ bits of the key. Consequently, for the 14-round **Blink-64**, only a 10-round distinguisher could potentially be useful to recover the full key. Thus, **Blink-64** is secure against differential attacks.

Similarly, for **Blink-128**, when the data does not exceed 2^{80} , it is highly unlikely to distinguish 10 rounds of **Blink-128**. The output of the third / fourth encryption or decryption round is affected by $(23 + 9 + 3) \times 4 = 140$ / $(31 + 23 + 9 + 3) \times 4 = 264$ bits of the key, respectively. Thus, **Blink-128** is also secure against differential attacks.

MT Differential Attacks. For **Blink-64** / **Blink-128**, h_1 and h_2 both belong to 2^{-64} -AXUHF / 2^{-128} -AXUHF, respectively. Therefore, under the data limitations of 2^{56} / 2^{80} , **Blink-64** and **Blink-128** are sufficiently resistant to MT differential attacks.

7.2 Impossible Differential Attacks

We explored the impossible differential trails for the center structure, considering r_1 -rounds of the upper half and r_2 -rounds of the lower half. Inspired by [52], we

constrained the differential model such that the input and output have only one active cell. By determining the feasibility of the model, we verify whether the input-output differential pair is valid. The final results are summarized in the Table 7, where ‘✓’ indicates the existence of an $r_1 + r_2$ impossible differential trail, and ‘blank’ denotes that for any difference pair with a single active cell in the input and output, a valid trail exists.

For **Blink-64**, the longest impossible differential trail spans 7 rounds, while for **Blink-128**, it spans 9 rounds. Since we have employed multiple different round keys, similar to differential analysis, these distinguishers are insufficient to pose a threat to the security of **Blink**.

MT Impossible Differential Attacks. Since h_1 and h_2 are 2^{-n} -AXUHF, when the difference of tweak is non-zero, the value of output difference is difficult to be determined. Therefore, we believe that **Blink** is secure against MT impossible differential attacks.

7.3 Linear Analysis

Since the diffusion matrix M is symmetric and involutory, and the shuffle layer is cell-based, Table 3 also presents the lower bounds on the number of active S-boxes for linear characteristics. Given that the linearity of the S-box is 8, its maximum squared correlation is 2^{-2} . Similar to differential analysis, we conclude that **Blink-64** and **Blink-128** are secure against linear attacks.

MT Linear Attacks. Given that for variable tweaks and non-zero λ_t , we have $c_h = 2^{-n}$, as defined in Equation 7, and by the linear analysis of E_{THF} , we conclude that **Blink** achieves security against MT linear attacks.

7.4 Integral Cryptanalysis

In [56], Todo introduced the division property as a generalization of the integral property, providing a powerful tool for cryptanalysis. To evaluate the division property of **Blink**, we formulated its propagation into SAT models [55] and utilized the open-source solver CaDiCaL [17] to solve the models. For **Blink-64**, our analysis identified that there exists at least one division trail over 7 rounds, propagating each input with 63 active bits to each output with 1 active bit. This indicates that the algebraic degree of each output bit after 7 rounds nearly reaches the theoretical maximum value of 63, as it is challenging to eliminate all terms of degree 63 through XOR operations. Similarly, for **Blink-128**, division trails were discovered over 9 rounds for each inputs consisting of 127 active bits.

These findings highlight that both **Blink-64** and **Blink-128** exhibit strong resistance against ST / MT integral attacks.

7.5 Meet-in-the-Middle Attacks

Thanks to the use of a long master key, any four consecutive round keys in **Blink** are independent. For **Blink-64**, the output of the third and fourth encryption or decryption rounds is influenced by 112 and 176 bits of the key, respectively. For **Blink-128**, the output of the third and fourth encryption or decryption rounds is influenced by 140 and 264 bits of the key, respectively. Therefore, we conclude that long-round meet-in-the-middle attacks are unlikely to be effective.

7.6 Invariant Subspace Attacks

In [13], Beierle *et al.* proposed a method for proving resistance against invariant attacks, including the invariant subspace attack [42,43,30] and the nonlinear invariant attack [57]. They found that the attack requires the existence of an invariant for the substitution layer whose linear space is invariant under the linear layer and contains all differences between the round keys.

In many ciphers, the round keys are derived in the form $\mathbf{rk}_i = \mathbf{k} \oplus \mathbf{rc}_i$. Since the differences of the round constants are known, the analysis can be reduced to studying the set of differences between the round constants, denoted as D . The smallest linear subspace invariant under the linear layer L is defined as

$$W_L(D) := \sum_{\mathbf{c} \in D} \langle L^i(\mathbf{c}), i \geq 0 \rangle = \sum_{\mathbf{c} \in D} W_L(\mathbf{c}),$$

where $L = P \circ M$ in **Blink**, and $W_L(D)$ can be computed by Algorithm 1.

However, **Blink** employs many distinct round keys, and the differences of many round keys are not fully known, resulting in a smaller set D . For **Blink**, D is defined as

$$D := \{\mathbf{rc}_i \oplus \mathbf{rc}'_i \mid i \in \{1, \dots, a + b\}\}.$$

For **Blink-64**, the dimension of $W_L(D)$ is 30, which appears insufficient to conclusively demonstrate resistance against invariant attacks. Given the presence of five round keys $\mathbf{rk}_i$, we addressed this limitation by randomly selecting four constants to add to D , simulating the effect of varying round keys. This simulation was performed 1000 times, and in every case, the dimension of $W_L(D)$ was at least 50. Then using the first approach described in [13], we confirmed that no non-trivial invariants existed across all experiments. Since each simulation explores only a 2^{-320} fraction of the total key space, and considering that the 128-bit random round keys derived from the hash function may further increase the dimension of $W_L(D)$, we conclude that it is highly unlikely that a weak key subspace larger than 2^{-128} exists that could enable an invariant attack.

In contrast, for **Blink-128a** and **Blink-128b**, the dimension of $W_L(D)$ is 124. Using the same approach from [13], we verified that no non-trivial invariants are present in these instances either.

8 Hardware Evaluation

This section evaluates the minimal latency of a fully unrolled **Blink** circuit in hardware and compares it against other low-latency ciphers, including AES [25], PRINCE [20], Midori [8], SPEEDY [44], QARMA [6], QARMAv2 [7], and Mantis [14]. Results, summarized in Table 4, were obtained by synthesizing fully unrolled circuits using NanGate 45nm, NanGate 15nm and ASAP 7nm libraries with Synopsys Design Compiler, optimized via four stages of the `compile_ultra` flow.

When implemented in hardware, **Blink**'s reflection structure allows for identical encryption and decryption circuits. Multiplexers select the appropriate key based on the encryption/decryption control signal. To minimize latency, we directly implemented the composite operation MK as shown in Figure 2. For the three round keys derived from the hash functions, we first computed $h_1(t)$ and $h_2(t)$, then applied the M operation to obtain $\widehat{h_1(t)}$ and $\widehat{h_2(t)}$. The final round keys $h(t)$ and $\widehat{h(t)}$ were computed as $h_1(t) \oplus h_2(t)$ and $\widehat{h_1(t)} \oplus \widehat{h_2(t)}$, respectively.

Among tweakable block ciphers with comparable security levels, **Blink** achieves the lowest latency. Even among conventional block ciphers, its latency remains competitive. Remarkably, **Blink**'s overall latency remains nearly unchanged even when the tweak length is doubled. In contrast, QARMAv2 requires four additional rounds to maintain security under a doubled tweak length, resulting in a significant increase in latency. These results highlight the capability of the THF mode to construct low-latency tweakable block ciphers.

9 Conclusion

This work introduces the THF mode, a framework for constructing low-latency tweakable block ciphers based on three hash functions. As a concrete instantiation, we proposed **Blink**, a cipher family optimized for memory encryption with a strong focus on latency. Hardware evaluations demonstrate that all **Blink** variants meet stringent low-latency goals, validating the practicality of the THF approach.

Looking forward, THF provides a promising direction for designing tweakable block ciphers across various low-latency scenarios. The choice and design of hash functions within this framework remain critical, directly influencing security against multi-tweak attacks. Future work may explore tailored hash function constructions to further enhance the adaptability and robustness of THF-based ciphers.

References

1. Abidin, A., Larsson, J.: New universal hash functions. In: Armknecht, F., Lucks, S. (eds.) Research in Cryptology - 4th Western European Workshop, WEWoRC 2011, Weimar, Germany, July 20-22, 2011, Revised Selected Papers. Lecture Notes in Computer Science, vol. 7242, pp. 99–108. Springer (2011). https://doi.org/10.1007/978-3-642-34159-5_7, https://doi.org/10.1007/978-3-642-34159-5_7

Table 4. Minimal latency of different block ciphers

Cipher	Block Size	Tweak Length	Security Level (D, T)	Minimal Latency (ns)		
				Nangate 45nm	Nangate 15nm	ASAP 7nm
PRINCE	64	-	$2^n, 2^{127-n}$	2.63	0.832	1.164
Midori64	64	-	$2^{128}, 2^{128}$	4.69	0.972	1.900
Midori128	128	-	$2^{128}, 2^{128}$	6.06	1.015	2.421
SPEEDY-6-192	192	-	$2^{128}, 2^{128}$	2.54	0.585	0.965
MANTIS	64	64	$2^n, 2^{126-n}$	3.28	0.964	1.450
QARMA-64- σ_0	64	64	$2^n, 2^{128-n}$	3.44	0.926	1.521
QARMAv2 ₇ -64	64	64	$2^{56}, 2^{128}$	3.53	0.847	1.576
QARMAv2 ₉ -64	64	128	$2^{56}, 2^{128}$	4.40	0.971	1.972
QARMAv2 ₉ -128	128	128	$2^{80}, 2^{128}$	4.51	0.972	2.064
QARMAv2 ₁₁ -128	128	256	$2^{80}, 2^{128}$	5.30	1.058	2.465
QARMAv2 ₁₃ -128	128	128	$2^{80}, 2^{256}$	6.10	1.236	2.855
QARMAv2 ₁₅ -128	128	256	$2^{80}, 2^{256}$	6.99	1.357	3.249
Blink-64a	64	64	$2^{56}, 2^{128}$	2.92	0.625	1.145
Blink-64b	64	128	$2^{56}, 2^{128}$	3.00	0.627	1.146
Blink-128a	128	128	$2^{80}, 2^{128}$	3.44	0.715	1.302
Blink-128b	128	256	$2^{80}, 2^{128}$	3.39	0.715	1.307
Blink-128A	128	128	$2^{80}, 2^{256}$	4.21	0.877	1.626
Blink-128B	128	256	$2^{80}, 2^{256}$	4.23	0.879	1.624

2. Alfarano, G., Beierle, C., Isobe, T., Kölbl, S., Leander, G.: Shiftrows alternatives for AES-like ciphers and optimal cell permutations for midori and skinny. IACR Trans. Symm. Cryptol. **2018**(2), 20–47 (2018). <https://doi.org/10.13154/tosc.v2018.i2.20-47>
3. AMD: Secure encrypted virtualization (SEV) (2019), <https://developer.amd.com/sev/>
4. Anand, R., Banik, S., Caforio, A., Ishikawa, T., Isobe, T., Liu, F., Minematsu, K., Rahman, M., Sakamoto, K.: Gleeok: A family of low-latency prfs and its applications to authenticated encryption. IACR Transactions on Cryptographic Hardware and Embedded Systems **2024**(2), 545–587 (2024)
5. ARM: Arm CCA Security Model (Aug 2021), rev 1.0, Document Number DEN0096
6. Avanzi, R.: The QARMA block cipher family. IACR Trans. Symm. Cryptol. **2017**(1), 4–44 (2017). <https://doi.org/10.13154/tosc.v2017.i1.4-44>
7. Avanzi, R., Banik, S., Dunkelman, O., Eichlseder, M., Ghosh, S., Nageler, M., Regazzoni, F., et al.: The QARMAv2 family of tweakable block ciphers. IACR Transactions on Symmetric Cryptology **2023** (2023)
8. Banik, S., Bogdanov, A., Isobe, T., Shibutani, K., Hiwatari, H., Akishita, T., Regazzoni, F.: Midori: A block cipher for low energy. In: Iwata, T., Cheon, J.H. (eds.) ASIACRYPT 2015, Part II. LNCS, vol. 9453, pp. 411–436. Springer, Heidelberg (Nov / Dec 2015). https://doi.org/10.1007/978-3-662-48800-3_17
9. Banik, S., Isobe, T., Liu, F., Minematsu, K., Sakamoto, K.: Orthros: A low-latency PRF. IACR Trans. Symm. Cryptol. **2021**(1), 37–77 (2021). <https://doi.org/10.46586/tosc.v2021.i1.37-77>

10. Bao, Z., Guo, C., Guo, J., Song, L.: TNT: How to tweak a block cipher. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020, Part II. LNCS, vol. 12106, pp. 641–673. Springer, Heidelberg (May 2020). https://doi.org/10.1007/978-3-030-45724-2_22
11. Bao, Z., Guo, J., Ling, S., Sasaki, Y.: Peigen – a platform for evaluation, implementation, and generation of S-boxes. IACR Trans. Symm. Cryptol. **2019**(1), 330–394 (2019). <https://doi.org/10.13154/tosc.v2019.i1.330-394>
12. Bariant, A., Baudrin, J., Leurent, G., Pernot, C., Perrin, L., Peyrin, T.: Fast AES-based universal hash functions and MACs featuring lemac and petitmac. IACR Trans. Symmetric Cryptol. **2024**(2), 35–67 (2024). <https://doi.org/10.46586/TOSC.V2024.I2.35-67>, <https://doi.org/10.46586/tosc.v2024.i2.35-67>
13. Beierle, C., Canteaut, A., Leander, G., Rotella, Y.: Proving resistance against invariant attacks: How to choose the round constants. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017, Part II. LNCS, vol. 10402, pp. 647–678. Springer, Heidelberg (Aug 2017). https://doi.org/10.1007/978-3-319-63715-0_22
14. Beierle, C., Jean, J., Kölbl, S., Leander, G., Moradi, A., Peyrin, T., Sasaki, Y., Sasdrich, P., Sim, S.M.: The SKINNY family of block ciphers and its low-latency variant MANTIS. In: Robshaw, M., Katz, J. (eds.) CRYPTO 2016, Part II. LNCS, vol. 9815, pp. 123–153. Springer, Heidelberg (Aug 2016). https://doi.org/10.1007/978-3-662-53008-5_5
15. Belkheyar, Y., Daemen, J., Dobraunig, C., Ghosh, S., Rasoolzadeh, S.: Bipbip: A low-latency tweakable block cipher with small dimensions. IACR Transactions on Cryptographic Hardware and Embedded Systems pp. 326–368 (2023)
16. Belkheyar, Y., Derbez, P., Ghosh, S., Leander, G., Mella, S., Perrin, L., Rasoolzadeh, S., Stennes, L., Sun, S., Van Assche, G., Vizár, D.: ChiLow and ChiChi: New constructions for code encryption. In: Fehr, S., Fouque, P.A. (eds.) Advances in Cryptology – EUROCRYPT 2025. pp. 212–243. Springer Nature Switzerland, Cham (2025)
17. Biere, A.: Cadical at the sat race 2019 (2019)
18. Biham, E., Shamir, A.: Differential cryptanalysis of DES-like cryptosystems. In: Menezes, A.J., Vanstone, S.A. (eds.) CRYPTO’90. LNCS, vol. 537, pp. 2–21. Springer, Heidelberg (Aug 1991). https://doi.org/10.1007/3-540-38424-3_1
19. Black, J., Halevi, S., Krawczyk, H., Krovetz, T., Rogaway, P.: UMAC: fast and secure message authentication. In: Wiener, M.J. (ed.) Advances in Cryptology - CRYPTO ’99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings. Lecture Notes in Computer Science, vol. 1666, pp. 216–233. Springer (1999). https://doi.org/10.1007/3-540-48405-1_14, https://doi.org/10.1007/3-540-48405-1_14
20. Borghoff, J., Canteaut, A., Güneysu, T., Kavun, E.B., Knežević, M., Knudsen, L.R., Leander, G., Nikov, V., Paar, C., Rechberger, C., Rombouts, P., Thomsen, S.S., Yalçın, T.: PRINCE - A low-latency block cipher for pervasive computing applications - extended abstract. In: Wang, X., Sako, K. (eds.) ASIACRYPT 2012. LNCS, vol. 7658, pp. 208–225. Springer, Heidelberg (Dec 2012). https://doi.org/10.1007/978-3-642-34961-4_14
21. Bozilov, D., Eichlseder, M., Knezevic, M., Lambin, B., Leander, G., Moos, T., Nikov, V., Rasoolzadeh, S., Todo, Y., Wiemer, F.: PRINCEv2 - more security for (almost) no overhead. In: Dunkelman, O., Jr., M.J.J., O’Flynn, C. (eds.) Selected Areas in Cryptography - SAC 2020 - 27th International Conference, Halifax, NS, Canada (Virtual Event), October 21-23, 2020, Revised Selected Papers. Lecture Notes in Computer Science, vol. 12804, pp. 483–511. Springer (2020).

- https://doi.org/10.1007/978-3-030-81652-0_19, https://doi.org/10.1007/978-3-030-81652-0_19
22. Canale, F., Güneysu, T., Leander, G., Thoma, J.P., Todo, Y., Ueno, R.: {SCARF}—a {Low-Latency} block cipher for secure {Cache-Randomization}. In: 32nd USENIX Security Symposium (USENIX Security 23). pp. 1937–1954 (2023)
 23. Carter, J.L., Wegman, M.N.: Universal classes of hash functions. *Journal of Computer and System Sciences* **18**(2), 143–154 (1979)
 24. Cary, M., Venkatesan, R.: A message authentication code based on unimodular matrix groups. In: Boneh, D. (ed.) *Advances in Cryptology - CRYPTO 2003*, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17–21, 2003, Proceedings. *Lecture Notes in Computer Science*, vol. 2729, pp. 500–512. Springer (2003). https://doi.org/10.1007/978-3-540-45146-4_29, https://doi.org/10.1007/978-3-540-45146-4_29
 25. Daemen, J., Rijmen, V.: The block cipher Rijndael. In: Quisquater, J., Schneier, B. (eds.) *Smart Card Research and Applications*, This International Conference, CARDIS '98, Louvain-la-Neuve, Belgium, September 14–16, 1998, Proceedings. *Lecture Notes in Computer Science*, vol. 1820, pp. 277–284. Springer (1998). https://doi.org/10.1007/10721064_26, https://doi.org/10.1007/10721064_26
 26. Daemen, J., Rijmen, V.: *The design of Rijndael*, vol. 2. Springer (2002)
 27. Flórez-Gutiérrez, A., Grassi, L., Leander, G., Sibleyras, F., Todo, Y.: General practical cryptanalysis of the sum of round-reduced block ciphers and ZIP-AES. In: Chung, K.M., Sasaki, Y. (eds.) *Advances in Cryptology – ASIACRYPT 2024*. pp. 280–311. Springer Nature Singapore, Singapore (2025)
 28. Ghosh, K., Fuchs, J., Amiri-Eliasi, P., Daemen, J.: Universal hashing based on field multiplication and (near-)MDS matrices. In: Mrabet, N.E., Feo, L.D., Duquesne, S. (eds.) *Progress in Cryptology - AFRICACRYPT 2023 - 14th International Conference on Cryptology in Africa*, Sousse, Tunisia, July 19–21, 2023, Proceedings. *Lecture Notes in Computer Science*, vol. 14064, pp. 129–150. Springer (2023). https://doi.org/10.1007/978-3-031-37679-5_6, https://doi.org/10.1007/978-3-031-37679-5_6
 29. Gueron, S.: Memory encryption for general-purpose processors. *IEEE Security & Privacy* **14**(6), 54–62 (2016). <https://doi.org/10.1109/MSP.2016.124>
 30. Guo, J., Jean, J., Nikolic, I., Qiao, K., Sasaki, Y., Sim, S.M.: Invariant subspace attack against Midori64 and the resistance criteria for S-box designs. *IACR Trans. Symm. Cryptol.* **2016**(1), 33–56 (2016). <https://doi.org/10.13154/tosc.v2016.i1.33-56>, <https://tosc.iacr.org/index.php/ToSC/article/view/534>
 31. Hoang, V.T., Krovetz, T., Rogaway, P.: Robust authenticated-encryption AEZ and the problem that it solves. In: Oswald, E., Fischlin, M. (eds.) *EUROCRYPT 2015*, Part I. LNCS, vol. 9056, pp. 15–44. Springer, Heidelberg (Apr 2015). https://doi.org/10.1007/978-3-662-46800-5_2
 32. Hu, K., Khairallah, M., Peyrin, T., Tan, Q.Q.: uknit: Breaking round-alignment for cipher design—featuring uknit-bc, an ultra low-latency block cipher. *Cryptology ePrint Archive* (2024)
 33. Intel: Intel trust domain extensions. Whitepaper (2020)
 34. Jean, J., Nikolic, I., Peyrin, T.: Tweaks and keys for block ciphers: The TWEAKEY framework. In: Sarkar, P., Iwata, T. (eds.) *ASIACRYPT 2014*, Part II. LNCS, vol. 8874, pp. 274–288. Springer, Heidelberg (Dec 2014). https://doi.org/10.1007/978-3-662-45608-8_15
 35. Jha, A., Khairallah, M., Nandi, M., Saha, A.: Tight security of TNT and beyond - attacks, proofs and possibilities for the cascaded LRW paradigm. In: Joye, M.,

- Leander, G. (eds.) *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 14651, pp. 249–279. Springer (2024). https://doi.org/10.1007/978-3-031-58716-0_9, https://doi.org/10.1007/978-3-031-58716-0_9
36. Jha, A., List, E., Minematsu, K., Mishra, S., Nandi, M.: XHX – a framework for optimally secure tweakable block ciphers from classical block ciphers and universal hashing. In: Lange, T., Dunkelman, O. (eds.) *Progress in Cryptology – LATINCRYPT 2017*. pp. 207–227. Springer International Publishing, Cham (2019)
 37. Jha, A., List, E., Minematsu, K., Mishra, S., Nandi, M.: XHX - A framework for optimally secure tweakable block ciphers from classical block ciphers and universal hashing. In: Lange, T., Dunkelman, O. (eds.) *LATINCRYPT 2017*. LNCS, vol. 11368, pp. 207–227. Springer, Heidelberg (Sep 2019). https://doi.org/10.1007/978-3-030-25283-0_12
 38. Jha, A., Nandi, M.: Tight security of cascaded LRW2. *Journal of Cryptology* **33**(3), 1272–1317 (Jul 2020). <https://doi.org/10.1007/s00145-020-09347-y>
 39. Krawczyk, H.: LFSR-based hashing and authentication. In: Desmedt, Y. (ed.) *Advances in Cryptology - CRYPTO '94*, 14th Annual International Cryptology Conference, Santa Barbara, California, USA, August 21-25, 1994, Proceedings. *Lecture Notes in Computer Science*, vol. 839, pp. 129–139. Springer (1994). https://doi.org/10.1007/3-540-48658-5_15, https://doi.org/10.1007/3-540-48658-5_15
 40. Landecker, W., Shrimpton, T., Terashima, R.S.: Tweakable blockciphers with beyond birthday-bound security. In: Safavi-Naini, R., Canetti, R. (eds.) *CRYPTO 2012*. LNCS, vol. 7417, pp. 14–30. Springer, Heidelberg (Aug 2012). https://doi.org/10.1007/978-3-642-32009-5_2
 41. Landecker, W., Shrimpton, T., Terashima, R.S.: Tweakable blockciphers with beyond birthday-bound security. In: Safavi-Naini, R., Canetti, R. (eds.) *Advances in Cryptology – CRYPTO 2012*. pp. 14–30. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)
 42. Leander, G., Abdelraheem, M.A., AlKhazimi, H., Zenner, E.: A cryptanalysis of PRINTcipher: The invariant subspace attack. In: Rogaway, P. (ed.) *CRYPTO 2011*. LNCS, vol. 6841, pp. 206–221. Springer, Heidelberg (Aug 2011). https://doi.org/10.1007/978-3-642-22792-9_12
 43. Leander, G., Minaud, B., Rønjom, S.: A generic approach to invariant subspace attacks: Cryptanalysis of robin, iSCREAM and Zorro. In: Oswald, E., Fischlin, M. (eds.) *EUROCRYPT 2015*, Part I. LNCS, vol. 9056, pp. 254–283. Springer, Heidelberg (Apr 2015). https://doi.org/10.1007/978-3-662-46800-5_11
 44. Leander, G., Moos, T., Moradi, A., Rasoolzadeh, S.: The SPEEDY family of block ciphers engineering an ultra low-latency cipher from gate level for secure processor architectures. *IACR TCHES* **2021**(4), 510–545 (2021). <https://doi.org/10.46586/tches.v2021.i4.510-545>, <https://tches.iacr.org/index.php/TCHES/article/view/9074>
 45. Lee, B., Lee, J.: Tweakable block ciphers secure beyond the birthday bound in the ideal cipher model. In: Peyrin, T., Galbraith, S. (eds.) *Advances in Cryptology – ASIACRYPT 2018*. pp. 305–335. Springer International Publishing, Cham (2018)
 46. Liskov, M., Rivest, R.L., Wagner, D.: Tweakable block ciphers. In: Yung, M. (ed.) *CRYPTO 2002*. LNCS, vol. 2442, pp. 31–46. Springer, Heidelberg (Aug 2002). https://doi.org/10.1007/3-540-45708-9_3
 47. Mansour, Y., Nisan, N., Tiwari, P.: The computational complexity of universal hashing. In: *22nd ACM STOC*. pp. 235–243. ACM Press (May 1990). <https://doi.org/10.1145/100216.100246>

48. McGrew, D.A., Viega, J.: The galois/counter mode of operation (GCM). Submission to NIST Modes of Operation Process (2004), <https://csrc.nist.gov/groups/ST/toolkit/BCM/documents/proposedmodes/gcm/gcm-spec.pdf>
49. Mennink, B., Neves, S.: Optimal PRFs from blockcipher designs. *IACR Trans. Symm. Cryptol.* **2017**(3), 228–252 (2017). <https://doi.org/10.13154/tosc.v2017.i3.228-252>
50. Nandi, M.: On the minimum number of multiplications necessary for universal hash functions. In: Cid, C., Rechberger, C. (eds.) *Fast Software Encryption - 21st International Workshop, FSE 2014, London, UK, March 3-5, 2014. Revised Selected Papers. Lecture Notes in Computer Science*, vol. 8540, pp. 489–508. Springer (2014). https://doi.org/10.1007/978-3-662-46706-0_25, https://doi.org/10.1007/978-3-662-46706-0_25
51. Rasoolzadeh, S.: Low-latency boolean functions and bijective s-boxes. *IACR Transactions on Symmetric Cryptology* **2022**(3), 403–447 (Sep 2022). <https://doi.org/10.46586/tosc.v2022.i3.403-447>, <https://tosc.iacr.org/index.php/ToSC/article/view/9862>
52. Sasaki, Y., Todo, Y.: New impossible differential search tool from design and cryptanalysis aspects - revealing structural properties of several ciphers. In: Coron, J.S., Nielsen, J.B. (eds.) *EUROCRYPT 2017, Part III. LNCS*, vol. 10212, pp. 185–215. Springer, Heidelberg (Apr / May 2017). https://doi.org/10.1007/978-3-319-56617-7_7
53. Shen, Y., Standaert, F.X., Wang, L.: Forgery attacks on several beyond-birthday-bound secure MACs. In: Guo, J., Steinfeld, R. (eds.) *Advances in Cryptology – ASIACRYPT 2023*. pp. 169–189. Springer Nature Singapore, Singapore (2023)
54. Stinson, D.R.: Universal hashing and authentication codes. In: Feigenbaum, J. (ed.) *CRYPTO’91. LNCS*, vol. 576, pp. 74–85. Springer, Heidelberg (Aug 1992). https://doi.org/10.1007/3-540-46766-1_5
55. Sun, L., Wang, W., Wang, M.: Automatic search of bit-based division property for ARX ciphers and word-based division property. In: Takagi, T., Peyrin, T. (eds.) *ASIACRYPT 2017, Part I. LNCS*, vol. 10624, pp. 128–157. Springer, Heidelberg (Dec 2017). https://doi.org/10.1007/978-3-319-70694-8_5
56. Todo, Y.: Structural evaluation by generalized integral property. In: Oswald, E., Fischlin, M. (eds.) *EUROCRYPT 2015, Part I. LNCS*, vol. 9056, pp. 287–314. Springer, Heidelberg (Apr 2015). https://doi.org/10.1007/978-3-662-46800-5_12
57. Todo, Y., Leander, G., Sasaki, Y.: Nonlinear invariant attack - practical attack on full SCREAM, iSCREAM, and Midori64. In: Cheon, J.H., Takagi, T. (eds.) *ASIACRYPT 2016, Part II. LNCS*, vol. 10032, pp. 3–33. Springer, Heidelberg (Dec 2016). https://doi.org/10.1007/978-3-662-53890-6_1
58. Wang, J., Huang, T., Wu, S., Liu, Z.: Twinkle: A family of low-latency schemes for authenticated encryption and pointer authentication. *IACR Communications in Cryptology* **1**(2) (2024)
59. Wegman, M.N., Carter, L.: New hash functions and their use in authentication and set equality. *J. Comput. Syst. Sci.* **22**(3), 265–279 (1981), [https://doi.org/10.1016/0022-0000\(81\)90033-7](https://doi.org/10.1016/0022-0000(81)90033-7)

A Supplementary Material for S-box Selection

The definition of extended bit permutation equivalence can be found in Definition 2. It is easy to verify that the properties of linearity, uniformity, algebraic degree, and full diffusion are the same within the same equivalence class.

Definition 2 (Extended Bit Permutation Equivalence [51]). *Two n -bit to m -bit Boolean functions f and g are called extended bit permutation equivalent if there exist a bit permutation of n bits P_i , a bit permutation of m bits P_o , $\alpha \in \{0, 1\}^n$, and $\beta \in \{0, 1\}^m$ in such a way that $g(x) = (P_o \circ f \circ P_i(x \oplus \alpha)) \oplus \beta$ for all $x \in \{0, 1\}^n$.*

The three representatives of the extended bit permutation equivalence classes of 4-bit S-boxes, characterized by a depth of 3.5, linearity of 8, uniformity of 4, and containing involutory S-boxes, are listed in Table 5.

Table 5. The three representatives of the equivalence classes of 4-bit S-boxes with depth 3.5, linearity 8, uniformity 4, and containing involutory S-boxes.

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$S_1(x)$	0	1	2	4	3	8	f	c	9	5	b	6	7	e	d	a
$S_2(x)$	0	1	2	3	4	8	c	9	7	f	e	b	5	d	6	a
$S_3(x)$	0	1	2	3	4	c	e	8	7	d	f	b	6	5	a	9

Using the PEIGEN tool [11], their algebraic normal forms are presented below. The results show that only the first class achieves full diffusion, with each output bit exhibiting the maximum algebraic degree of 3.

Table 6. The algebraic normal forms of the three representatives.

$S_1(x)$	$y_0 = x_0 \oplus x_2 \oplus x_3 \oplus x_0x_1 \oplus x_0x_3 \oplus x_2x_3 \oplus x_0x_1x_2 \oplus x_0x_2x_3$
	$y_1 = x_1 \oplus x_2 \oplus x_0x_1 \oplus x_0x_2 \oplus x_1x_2 \oplus x_0x_1x_2 \oplus x_0x_1x_3 \oplus x_0x_2x_3 \oplus x_1x_2x_3$
	$y_2 = x_0x_1 \oplus x_0x_3 \oplus x_1x_2 \oplus x_2x_3 \oplus x_0x_1x_2 \oplus x_0x_1x_3 \oplus x_0x_2x_3 \oplus x_1x_2x_3$
	$y_3 = x_3 \oplus x_0x_2 \oplus x_0x_3 \oplus x_1x_2 \oplus x_2x_3 \oplus x_0x_1x_2 \oplus x_0x_2x_3$
$S_2(x)$	$y_0 = x_0 \oplus x_3 \oplus x_0x_2 \oplus x_0x_3 \oplus x_1x_3 \oplus x_0x_1x_2 \oplus x_0x_1x_3 \oplus x_0x_2x_3$
	$y_1 = x_1 \oplus x_3 \oplus x_1x_2 \oplus x_1x_3 \oplus x_2x_3$
	$y_2 = x_2 \oplus x_3 \oplus x_0x_2 \oplus x_2x_3 \oplus x_0x_1x_3 \oplus x_0x_2x_3$
	$y_3 = x_0x_2 \oplus x_0x_3 \oplus x_1x_2 \oplus x_1x_3 \oplus x_0x_1x_2 \oplus x_0x_1x_3 \oplus x_0x_2x_3$
$S_3(x)$	$y_0 = x_0 \oplus x_3 \oplus x_0x_2 \oplus x_0x_3 \oplus x_2x_3$
	$y_1 = x_1 \oplus x_3 \oplus x_0x_3 \oplus x_1x_3 \oplus x_0x_1x_2 \oplus x_0x_1x_3$
	$y_2 = x_2 \oplus x_3 \oplus x_2x_3 \oplus x_0x_1x_2 \oplus x_0x_1x_3 \oplus x_1x_2x_3$
	$y_3 = x_0x_2 \oplus x_0x_3 \oplus x_1x_2 \oplus x_1x_3 \oplus x_0x_1x_2 \oplus x_0x_1x_3 \oplus x_1x_2x_3$

B The Proof of 3-round Diffusion

Theorem 3. *Conditions 1 and 2 ensure that, in both the forward and backward directions, each output cell after three rounds is influenced by exactly 23 input cells.*

Proof. Let the input be denoted by $\mathbf{x}_0$, the output of round i by $\mathbf{x}_i$, and the input to the shuffle layer in round i by $\mathbf{y}_i$. Clearly, $\mathbf{x}_{i+1}$ is the output of the shuffle layer in round i . We prove the forward case; the backward case follows analogously.

Due to the diffusion matrix M , each cell of $\mathbf{x}_i$ is influenced by 3 cells from a specific column of $\mathbf{x}_{i-1}$. Moreover, if two cells in $\mathbf{x}_i$ are influenced by cells from the same column of $\mathbf{x}_{i-1}$, they are collectively influenced by all 4 cells of that column.

According to Condition 1, each column of $\mathbf{y}_2$ is influenced by 12 distinct cells from 4 columns of $\mathbf{x}_1$, with each column contributing 3 cells. For convenience, we group these 3 cells from the same column into one group. Each cell within this column of $\mathbf{y}_2$ is then influenced by cells from three of these groups, resulting in a total of 9 distinct cells of $\mathbf{x}_1$. Condition 2 ensures that any three of these four groups originate from at least 7 distinct columns of $\mathbf{x}_0$. Upon verification, exactly two pairs of these cells come from the same column; otherwise, Condition 2 would be violated. Therefore, each cell of $\mathbf{y}_2$ is influenced by $5 \times 3 + 2 \times 4 = 23$ cells from $\mathbf{x}_0$.

Since the shuffle layer itself does not introduce additional mixing, each cell of $\mathbf{x}_3$ is influenced by 23 cells from $\mathbf{x}_0$.

C Supplementary Material for Security Analysis

Table 7. Existence of $r_1 + r_2$ -round impossible difference

r_2 / r_1	Blink-64								Blink-128									
	0	1	2	3	4	5	6		0	1	2	3	4	5	6	7	8	9
0	✓	✓	✓	✓	✓	✓	✓		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
1	✓	✓	✓	✓	✓	✓			✓	✓	✓	✓	✓	✓	✓	✓	✓	
2	✓	✓	✓	✓	✓	✓			✓	✓	✓	✓	✓	✓	✓	✓		
3	✓	✓	✓	✓					✓	✓	✓	✓	✓	✓	✓			
4	✓	✓	✓						✓	✓	✓	✓	✓	✓				
5	✓	✓	✓						✓	✓	✓	✓	✓	✓				
6	✓								✓	✓	✓	✓	✓					
7									✓	✓	✓	✓						
8									✓	✓								
9									✓									

Algorithm 1 Computing $W_L(D)$

Require: list of differences D , linear layer L as a matrix

Ensure: the subspace $W_L(D)$

```
1:  $R \leftarrow$  an empty list
2:  $k \leftarrow$  the multiplicative order of  $L$ 
3: for all  $c$  in  $D$  do
4:   for all  $j$  from 0 to  $k - 1$  do
5:     Add  $L^j(c)$  into  $R$ 
6:   end for
7: end for
8:
9: return the span of  $R$ 
```

D Round Constants

The round constants in 64-bit block version are detailed as follows.

$$\begin{aligned}rc_1 &= 0x13198a2e03707344. \\rc_2 &= 0x082efa98ec4e6c89. \\rc_3 &= 0xbe5466cf34e90c6c. \\rc_4 &= 0x3f84d5b5b5470917. \\rc_5 &= 0xd1310ba698dfb5ac. \\rc'_1 &= 0x0d95748f728eb658. \\rc'_2 &= 0x7b54a41dc25a59b5. \\rc'_3 &= 0xc5d1b023286085f0. \\rc'_4 &= 0x8e79dcb0603a180e. \\rc'_5 &= 0xd71577c1bd314b27.\end{aligned}$$

The round constants in 128-bit block version are detailed as follows.

$rc_1 = 0x243f6a8885a308d313198a2e03707344.$
 $rc_2 = 0xa4093822299f31d0082efa98ec4e6c89.$
 $rc_3 = 0x452821e638d01377be5466cf34e90c6c.$
 $rc_4 = 0xc0ac29b7c97c50dd3f84d5b5b5470917.$
 $rc_5 = 0x9216d5d98979fb1bd1310ba698dfb5ac.$
 $rc_6 = 0x2ffd72dbd01adfb7b8e1afed6a267e96.$
 $rc_7 = 0xba7c9045f12c7f9924a19947b3916cf7.$
 $rc_8 = 0x0801f2e2858efc16636920d871574e69.$
 $rc'_1 = 0xa458fea3f4933d7e0d95748f728eb658.$
 $rc'_2 = 0x718bcd5882154aee7b54a41dc25a59b5.$
 $rc'_3 = 0x9c30d5392af26013c5d1b023286085f0.$
 $rc'_4 = 0xca417918b8db38ef8e79dc0603a180e.$
 $rc'_5 = 0x6c9e0e8bb01e8a3ed71577c1bd314b27.$
 $rc'_6 = 0x78af2fda55605c60e65525f3aa55ab94.$
 $rc'_7 = 0x5748986263e8144055ca396a2aab10b6.$
 $rc'_8 = 0xb4cc5c341141e8cea15486af7c72e993.$

E Area comparison

Table 8 presents the area comparison of different block ciphers synthesized for minimal latency.

F Test Vector

F.1 Blink-64a

$m = 0x0$

$k = 0xd6a102d888a467e4d1d7dec33a246943e07c1dc6f302c57e762c2df9de6f0d21$
 $6dd387874a0b52ce3022e0ad78c78a0697779021b38e7fa1$

$t = 0x0123456789abcdef$

$c = 0xa4a0d10502be846e$

Table 8. Area comparison of various block ciphers optimized for minimum latency synthesis

Cipher	Block Size	Tweak Length	Security Level (D, T)	Area (μm^2)		
				Nangate 45nm	Nangate 15nm	ASAP 7nm
PRINCE	64	-	$2^n, 2^{127-n}$	8617.34	2005.75	859.55
Midori64	64	-	$2^{128}, 2^{128}$	17588.40	4290.43	1599.44
Midori128	128	-	$2^{128}, 2^{128}$	39959.56	12385.91	3945.64
SPEEDY-6-192	192	-	$2^{128}, 2^{128}$	26102.05	7173.69	2520.66
MANTIS	64	64	$2^n, 2^{126-n}$	11956.97	2794.49	1059.34
QARMA-64- σ_0	64	64	$2^n, 2^{128-n}$	13656.44	3162.88	1208.10
QARMAv2 ₇ -64	64	64	$2^{56}, 2^{128}$	14336.07	3406.09	1376.29
QARMAv2 ₉ -64	64	128	$2^{56}, 2^{128}$	16951.65	4157.28	1651.78
QARMAv2 ₉ -128	128	128	$2^{80}, 2^{128}$	33289.37	8668.84	3318.63
QARMAv2 ₁₁ -128	128	256	$2^{80}, 2^{128}$	39644.64	11362.12	3886.42
QARMAv2 ₁₃ -128	128	128	$2^{80}, 2^{256}$	46240.38	12849.91	4418.31
QARMAv2 ₁₅ -128	128	256	$2^{80}, 2^{256}$	52022.15	15443.76	4987.45
Blink-64a	64	64	$2^{56}, 2^{128}$	33431.41	9106.88	3006.10
Blink-64b	64	128	$2^{56}, 2^{128}$	54178.35	14744.03	4685.21
Blink-128a	128	128	$2^{80}, 2^{128}$	112327.81	30000.32	9486.08
Blink-128b	128	256	$2^{80}, 2^{128}$	199273.10	52152.29	15955.56
Blink-128A	128	128	$2^{80}, 2^{256}$	119455.55	31828.57	10140.54
Blink-128B	128	256	$2^{80}, 2^{256}$	206214.90	53980.79	16654.85

F.2 Blink-64b

$m = 0x0$

$k = 0xd6a102d888a467e4d1d7dec33a246943e07c1dc6f302c57e762c2df9de6f0d21$
 $6dd387874a0b52ce3022e0ad78c78a0697779021b38e7fa1$

$t = 0x0123456789abcdef0123456789abcdef$

$c = 0x743e142f17caaae1$

F.3 Blink-128a

$m = 0x0$

$k = 0xd6a102d888a467e4d1d7dec33a246943e07c1dc6f302c57e762c2df9de6f0d21$
6dd387874a0b52ce3022e0ad78c78a0697779021b38e7fa15e2b66350517f80f
2961c648d578bae174d70cb769c30a45cc40300fe8a342ca57a0bd0251ae39b6
21b8f104904374bbd6a102e234a664e421b8f104904374bbd6a102d888a666e4

$t = 0x0123456789abcdef0123456789abcdef$

$c = 0x713fc1546d924bf9cb4e96812eeff9ac$

F.4 Blink-128b

$m = 0x0$

$k = 0xd6a102d888a467e4d1d7dec33a246943e07c1dc6f302c57e762c2df9de6f0d21$
6dd387874a0b52ce3022e0ad78c78a0697779021b38e7fa15e2b66350517f80f
2961c648d578bae174d70cb769c30a45cc40300fe8a342ca57a0bd0251ae39b6
21b8f104904374bbd6a102e234a664e421b8f104904374bbd6a102d888a666e4

$t = 0x0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef$

$c = 0xc5c9de4a6384e9c69cb02ffbe7e3ccc0$

F.5 Blink-128A

$m = 0x0$

$k = 0xd6a102d888a467e4d1d7dec33a246943e07c1dc6f302c57e762c2df9de6f0d21$
6dd387874a0b52ce3022e0ad78c78a0697779021b38e7fa15e2b66350517f80f
2961c648d578bae174d70cb769c30a45cc40300fe8a342ca57a0bd0251ae39b6
21b8f104904374bbd6a102e234a664e421b8f104904374bbd6a102d888a666e4
28962a4c96893eda752c17026a6395c2d6963be43b2fc10813d73f5a4a48d28d

$t = 0x0123456789abcdef0123456789abcdef$

$c = 0xa227f9dd85e12a8a6ce9dcc03730bbd8$

F.6 Blink-128B

$m = 0x0$

$k = 0xd6a102d888a467e4d1d7dec33a246943e07c1dc6f302c57e762c2df9de6f0d21$
6dd387874a0b52ce3022e0ad78c78a0697779021b38e7fa15e2b66350517f80f
2961c648d578bae174d70cb769c30a45cc40300fe8a342ca57a0bd0251ae39b6
21b8f104904374bbd6a102e234a664e421b8f104904374bbd6a102d888a666e4
28962a4c96893eda752c17026a6395c2d6963be43b2fc10813d73f5a4a48d28d
 $t = 0x0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef$

$c = 0xf13aa94c65e193cc095970ef1e22f65e$